Linear regression

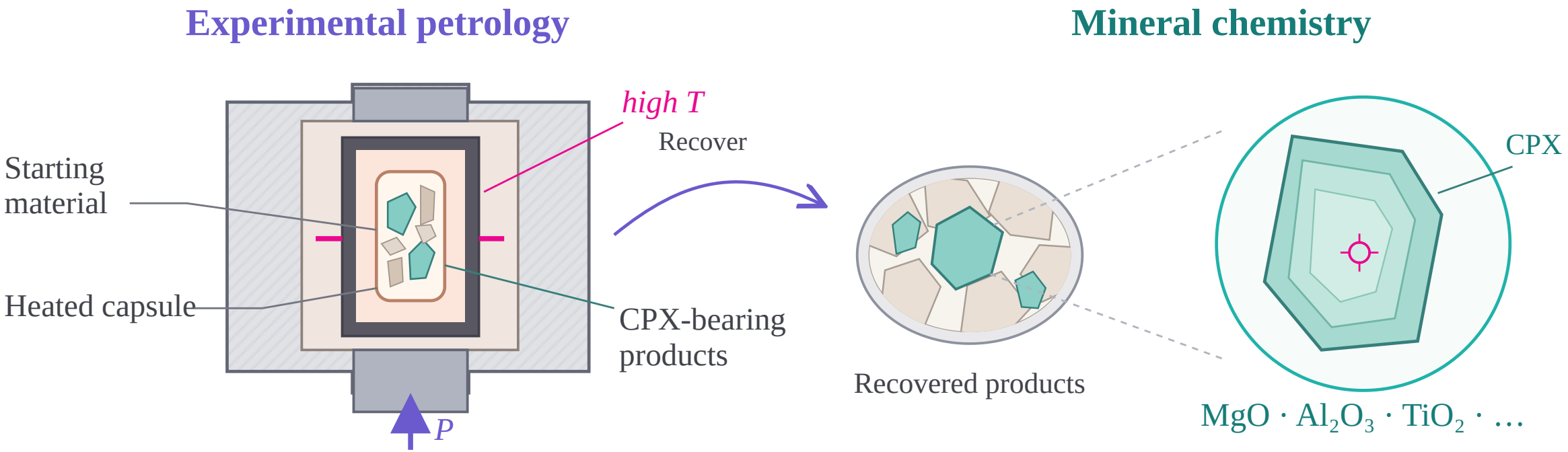
Experimental clinopyroxene temperature

Sébastien Biass
Earth Sciences, University of Geneva

Stéphane Guerrier
Earth Sciences & Pharmaceutical Sciences, University of Geneva

Samuel Orso
Earth Sciences & Pharmaceutical Sciences, University of Geneva

From controlled experiments to temperature prediction



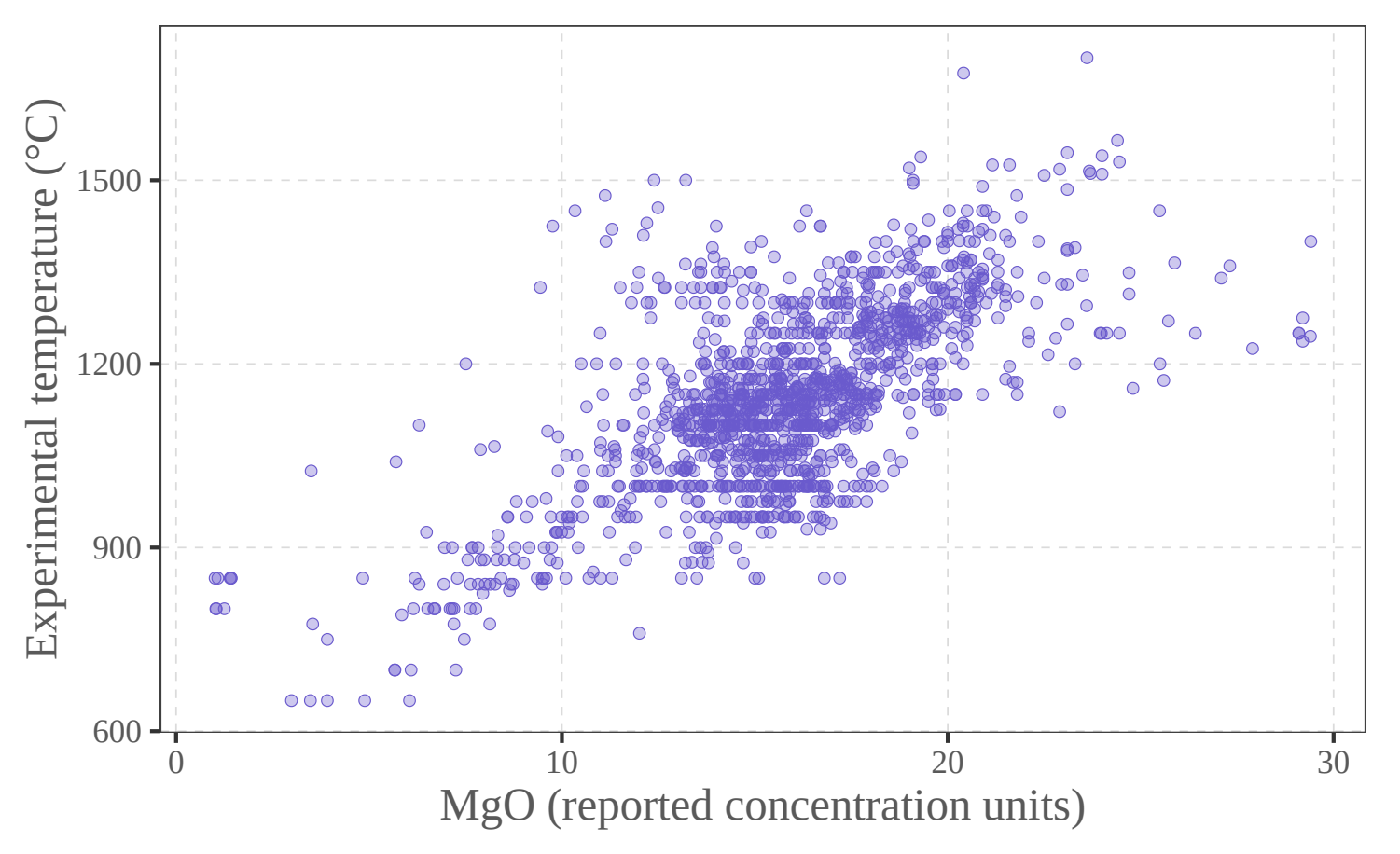
Illustrative piston-cylinder setup; experimental methods vary among studies.

Physical idea: Clinopyroxene chemistry contains information about its formation conditions, including temperature and pressure.

Our data: Experimental studies provide known temperatures and measured mineral compositions.

Question for this class: Can we predict experimental temperature from clinopyroxene chemistry?

Motivating example: experimental clinopyroxene



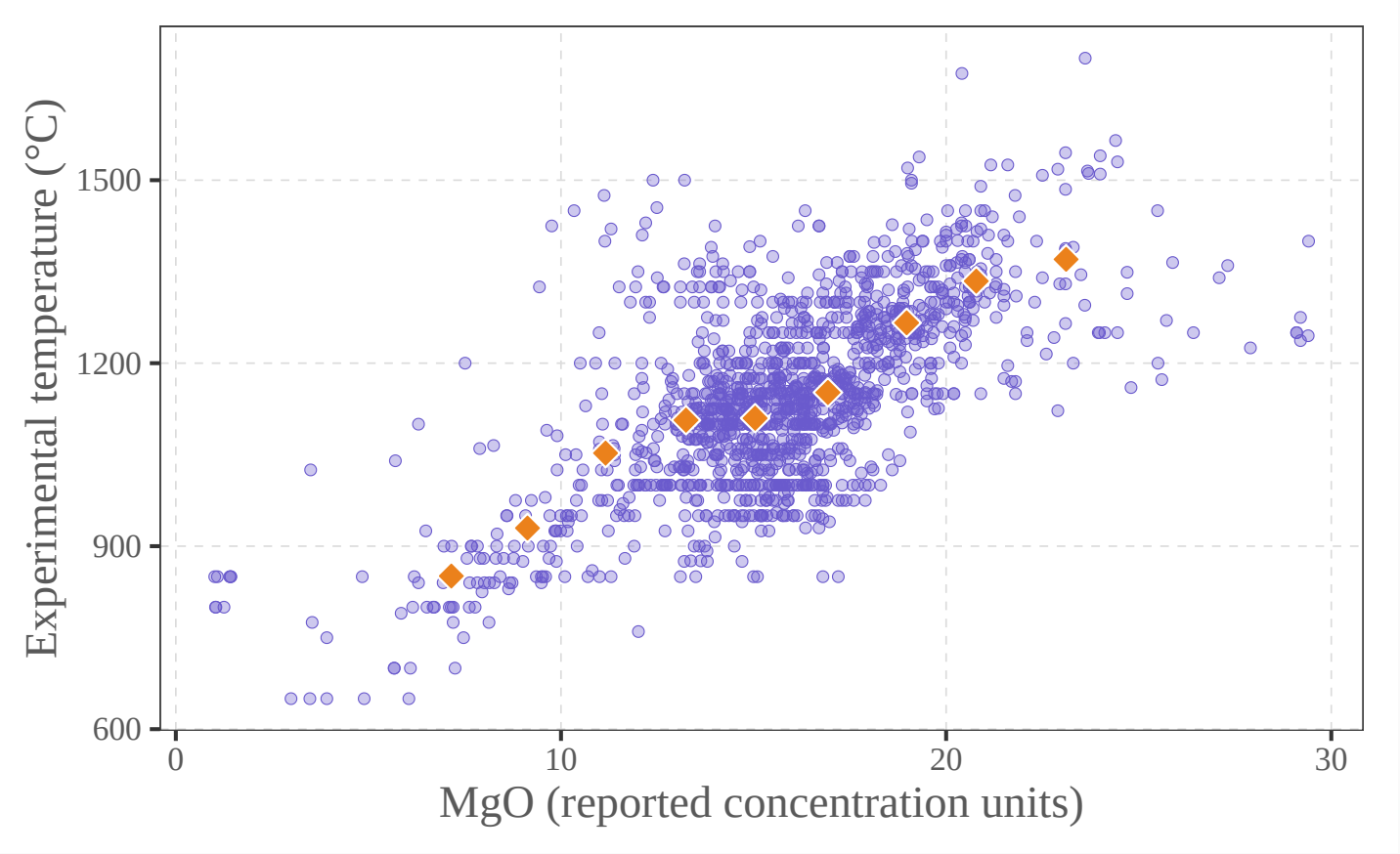
Each point is a reported CPX analysis with a known experimental temperature.

Can chemistry help predict temperature?

Regression describes the average relationship and the variation around it.

1,402 training observations. Oxides retain their reported units; no normalization is applied.

Regression models a conditional mean



$$T_i = m(x_i) + \varepsilon_i$$

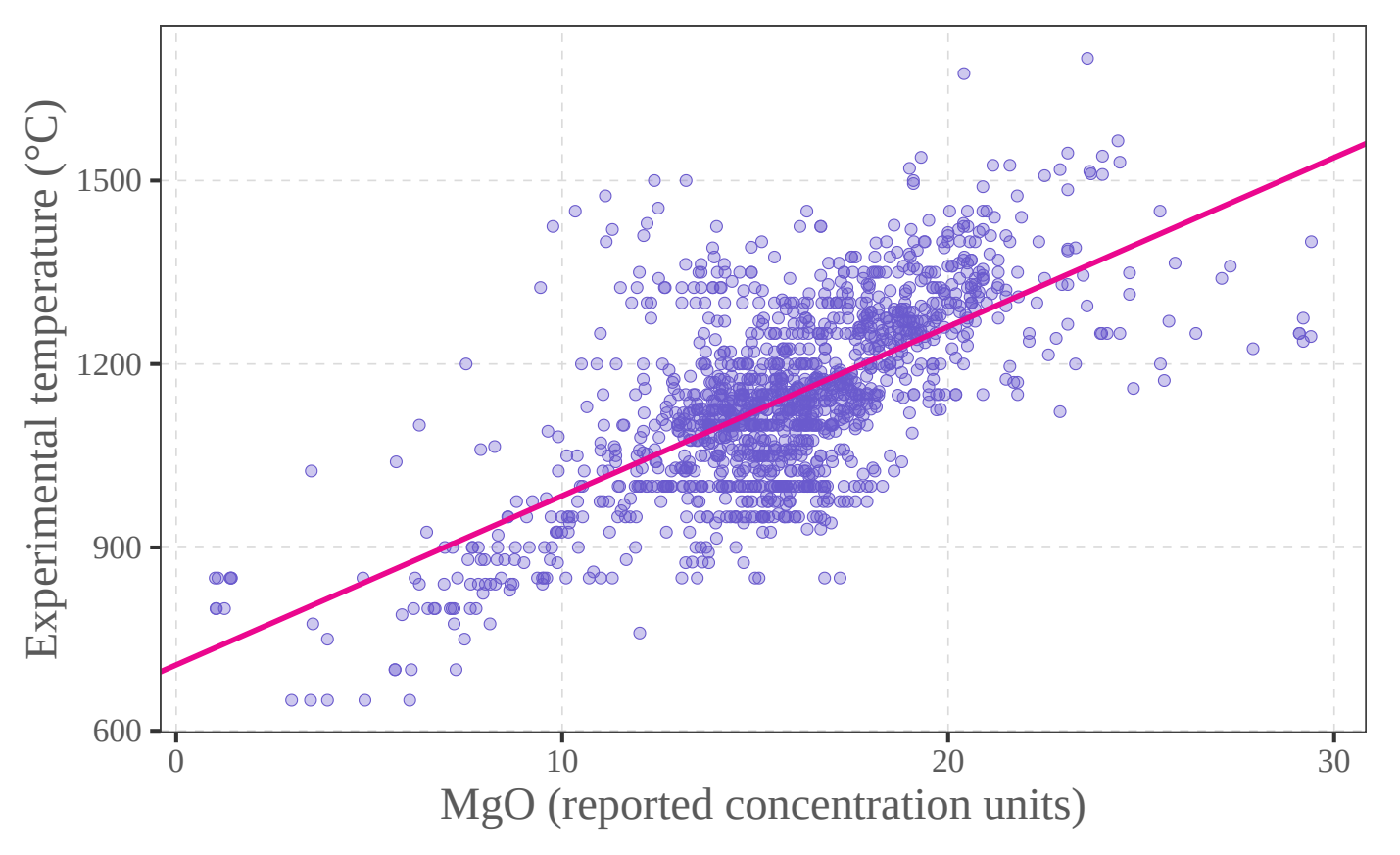
$$m(x) = \mathbb{E}[T \mid X = x]$$

Average temperature among analyses with a given MgO value.

Diamonds: averages in 2-unit MgO bins with ≥15 training records. They illustrate a mean; they are not the exact conditional expectation.

For the true conditional mean, $\mathbb{E}[\varepsilon \mid X] = 0$.

Linear regression: a straight-line mean

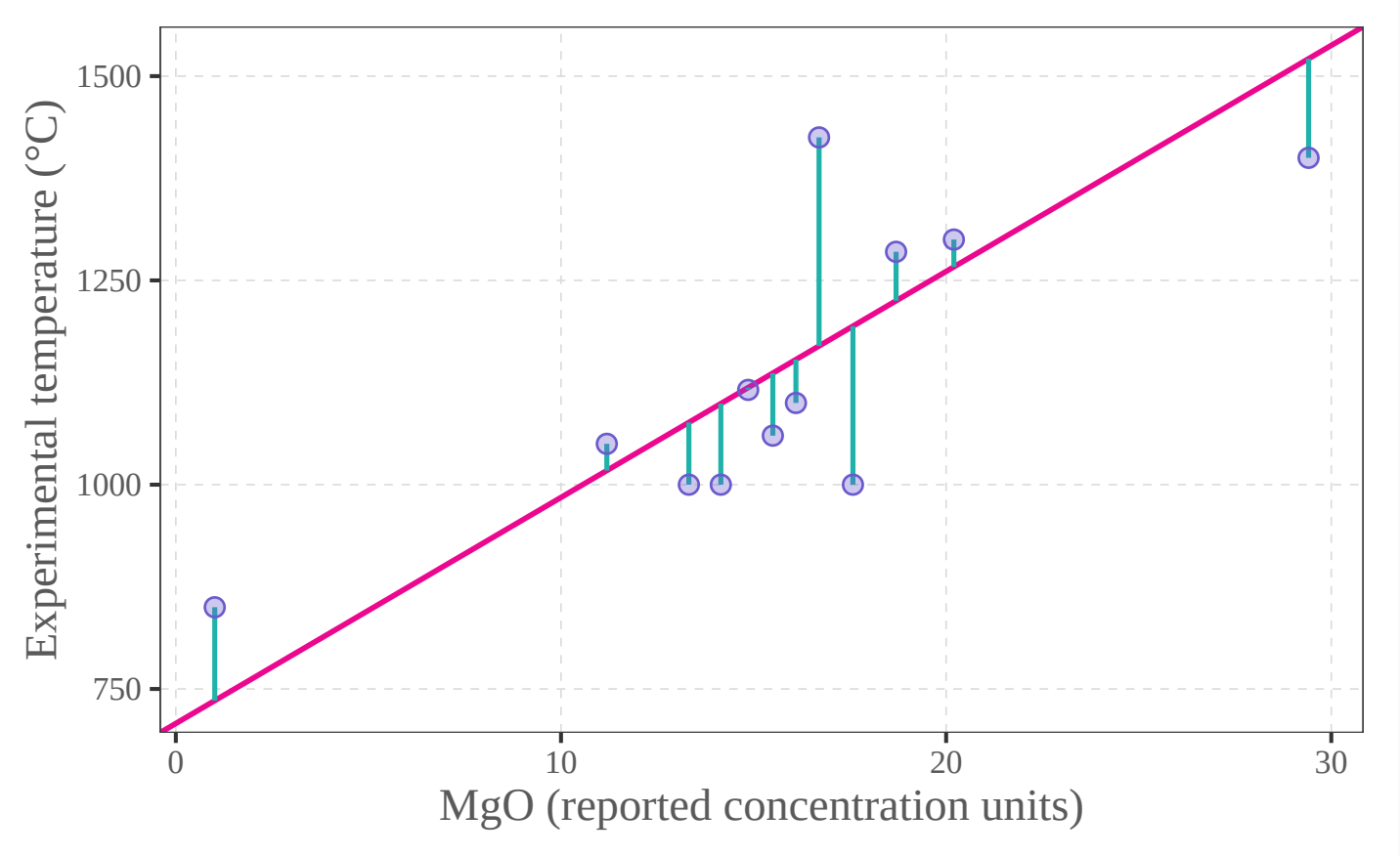


$$\mathbb{E}[T \mid X = x] \approx \beta_0 + \beta_1 x$$

β_0: intercept. β_1: slope.

The fitted line estimates the average temperature; observations still scatter around it.

Least squares: choose the line



12 training records shown for clarity; all 1,402 determine the line.

For any candidate line $b_0 + b_1 x$:

$$S(b_0, b_1) = \sum_i (T_i - b_0 - b_1 x_i)^2.$$

Choose $\hat{\beta}_0, \hat{\beta}_1$ to minimize S. These are called the **ordinary least squares (OLS) estimators**.

The vertical distances from observations to the fitted line are residuals:

$$e_i = T_i - \hat{T}_i.$$

Squaring prevents cancellation and penalizes large discrepancies more strongly.

Fit a simple regression in Python

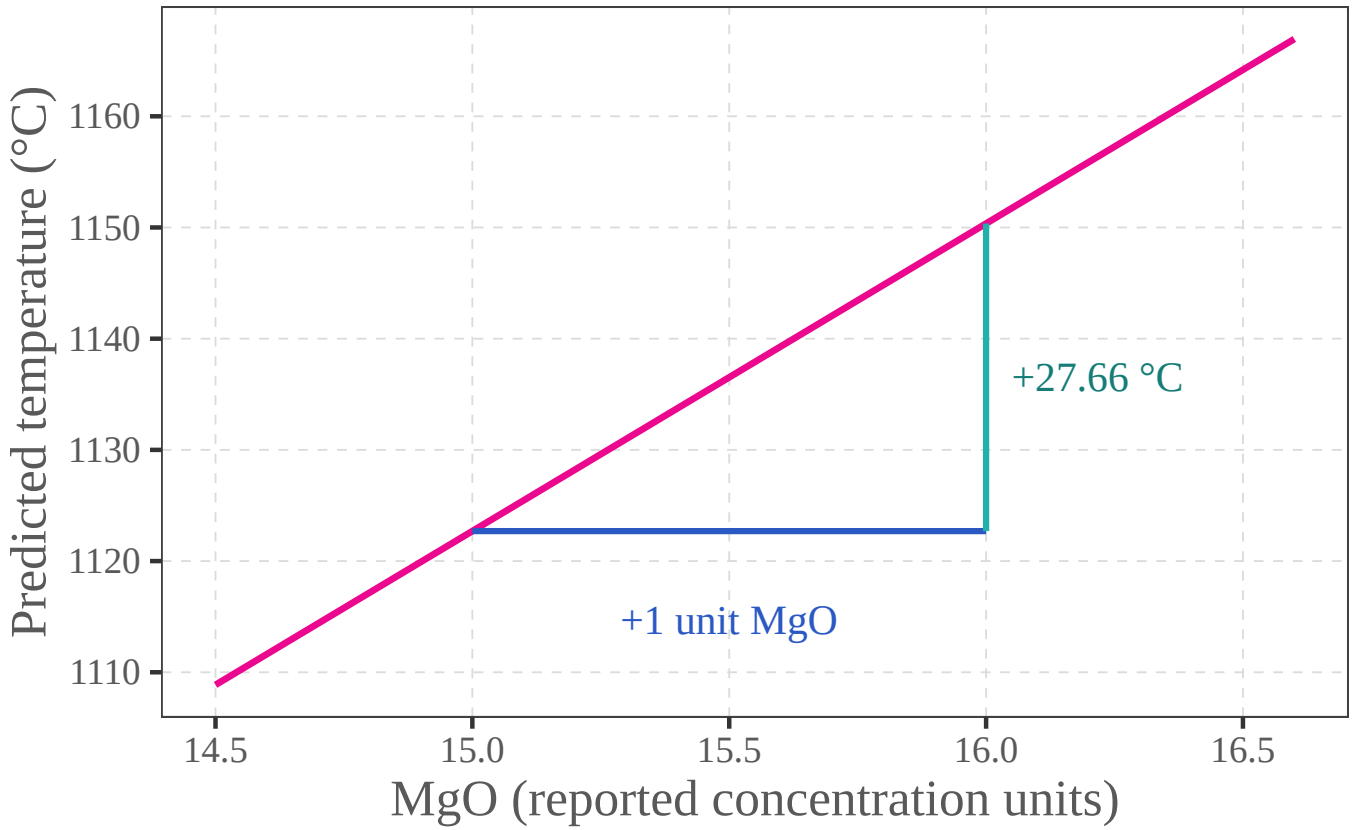
Fit the straight-line model using the supplied training observations.

```
from cpx_lesson import load_data
import statsmodels.formula.api as smf

train, _ = load_data()
simple = smf.ols("T_C ~ MgO_Cpx", data=train).fit()
print(simple.params.round(2))
```

`T_C ~ MgO_Cpx` specifies the response and predictor; `.fit()` estimates the OLS coefficients.

What does a coefficient mean?



$$\hat{T} = 707.80 + 27.66\,\mathrm{MgO}$$

One more unit of reported MgO corresponds to **27.66 °C higher predicted mean temperature** in this simple model.

This is an association, not the effect of adding MgO to an experiment.

The intercept is the line's prediction at MgO = 0. Its algebraic role does not establish a meaningful CPX composition.

Summarize the fitted regression

Use `simple.summary()` to inspect the fitted model; `slim=True` gives a compact version.

```
from cpx_lesson import load_data
import statsmodels.formula.api as smf
from IPython.display import display

train, _ = load_data()
simple = smf.ols(
    "T_C ~ MgO_Cpx", data=train
).fit()
display(simple.summary(slim=True))
```

OLS Regression Results

Dep. Variable:	T_C	R-squared:	0.447
Model:	OLS	Adj. R-squared:	0.447
No. Observations:	1402	F-statistic:	1133.
Covariance Type:	nonrobust	Prob (F-statistic):	1.78e-182

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	707.8019	13.286	53.274	0.000	681.739	733.865
MgO_Cpx	27.6596	0.822	33.660	0.000	26.048	29.272

Notes:
[1] The coefficient standard errors assume constant error variance and uncorrelated errors across observations.

Is the MgO slope different from zero?

Two-sided hypotheses

$$H_0 : \beta_1 = 0$$

No linear change in mean temperature with MgO in this simple model.

$$H_1 : \beta_1 \neq 0$$

The MgO slope is non-zero: positive or negative.

MgO in our fitted regression

$\hat{\beta}_1$	Standard error	t	p
27.66	0.82	33.66	< 0.001

If H_0 and the model assumptions hold, the **p-value** is the probability of a t statistic at least as far from zero as the one observed.

`0.000` in the summary is rounded; the p-value is not zero.

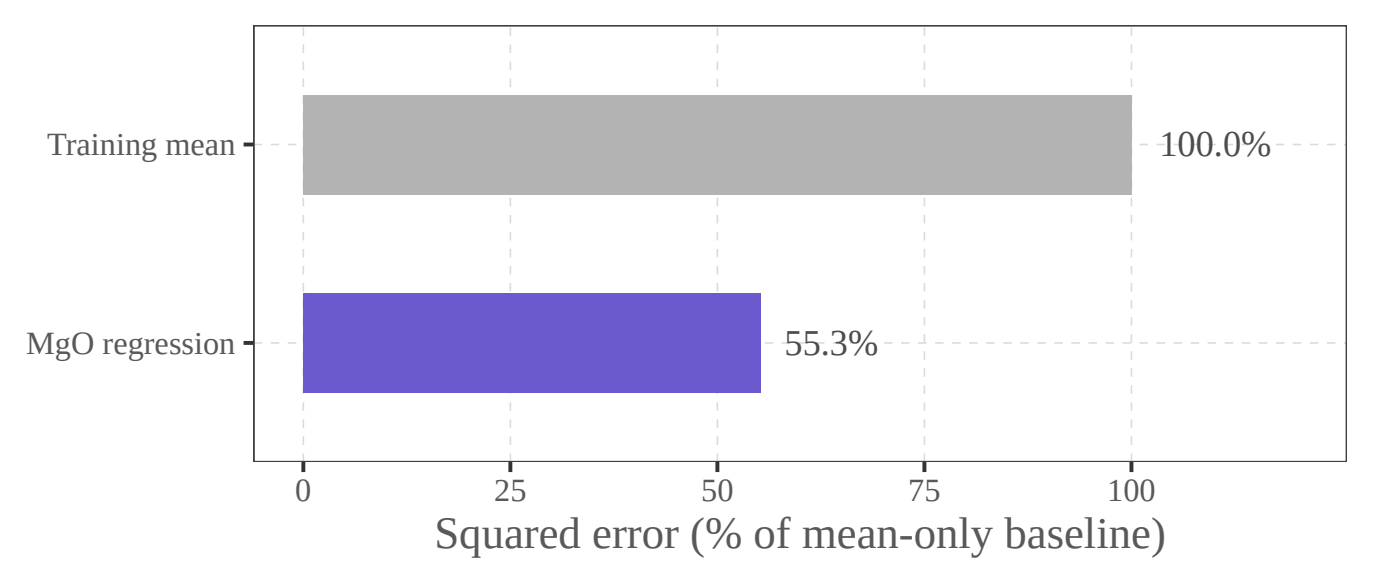
> At the 5% level, under the OLS assumptions, reject H_0: the data support a **positive linear association** between MgO and experimental temperature.

A p-value is not the probability that H_0 is true. A small p-value does not measure effect size, prove causation, or establish prediction accuracy.

R-squared: definition and interpretation

$$R^2 = 1 - \frac{\sum_{i \in \text{train}} (T_i - \hat{T}_i)^2}{\sum_{i \in \text{train}} (T_i - \bar{T}_{\text{train}})^2}.$$

T_i: measured temperature; $\hat{T}_i$: fitted temperature; $\bar{T}_{\text{train}}$: mean training temperature.



Here, $R^2 = 0.447$.

The fitted MgO relationship captures **44.7% of the variation in experimental training temperatures**.

This is a 44.7% reduction in squared error relative to predicting the training mean.

R^2 describes training fit. Check prediction accuracy on test observations.

Fit a multiple regression in Python

Fit temperature using MgO, Al_2O_3, and TiO_2 from the training observations.

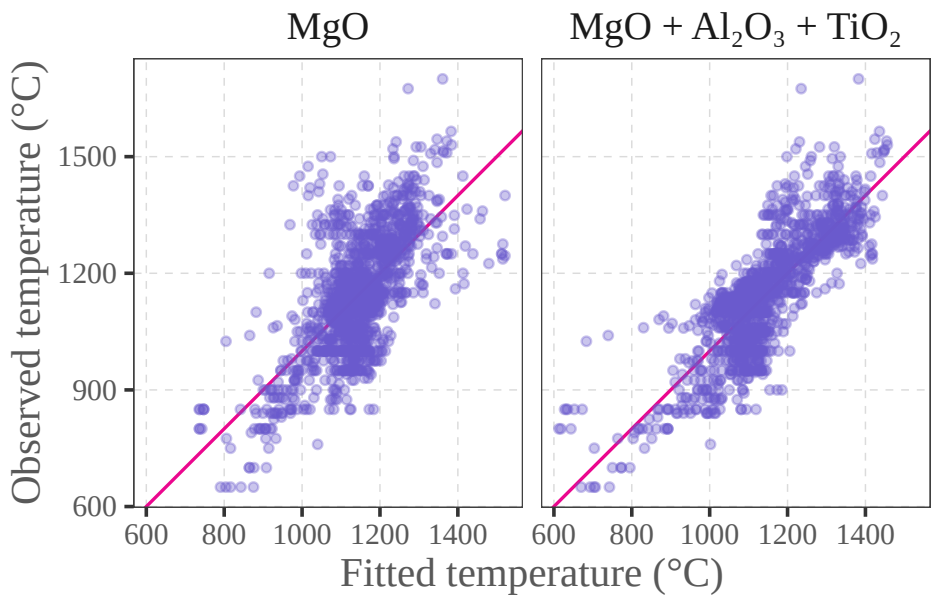
```
from cpx_lesson import load_data
import statsmodels.formula.api as smf

train, _ = load_data()
multiple = smf.ols(
    "T_C ~ MgO_Cpx + Al2O3_Cpx + TiO2_Cpx",
    data=train
).fit()
print(multiple.params.round(2))
```

The `+` signs include all three predictors; `.fit()` estimates the OLS coefficients.

Multiple regression: more chemical information

$$\hat{T} = 576.32 + 28.37\,\mathrm{MgO} + 25.10\,\mathrm{Al_2O_3} - 18.62\,\mathrm{TiO_2}$$



Training R^2: 0.447 → 0.672. Pink line: observed = fitted.

Each slope holds the other included predictors fixed.

Chemical predictors are correlated; coefficients are not independent chemical interventions.

R^2 summarizes training fit. Adding predictors cannot lower it on the same rows, but need not improve future predictions.

Summarize the multiple regression

Use `multiple.summary(slim=True)` to inspect the three-oxide model.

```
from cpx_lesson import load_data
import statsmodels.formula.api as smf
from IPython.display import display

train, _ = load_data()
multiple = smf.ols(
    "T_C ~ MgO_Cpx + Al2O3_Cpx"
    " + TiO2_Cpx", data=train
).fit()
display(multiple.summary(slim=True))
```

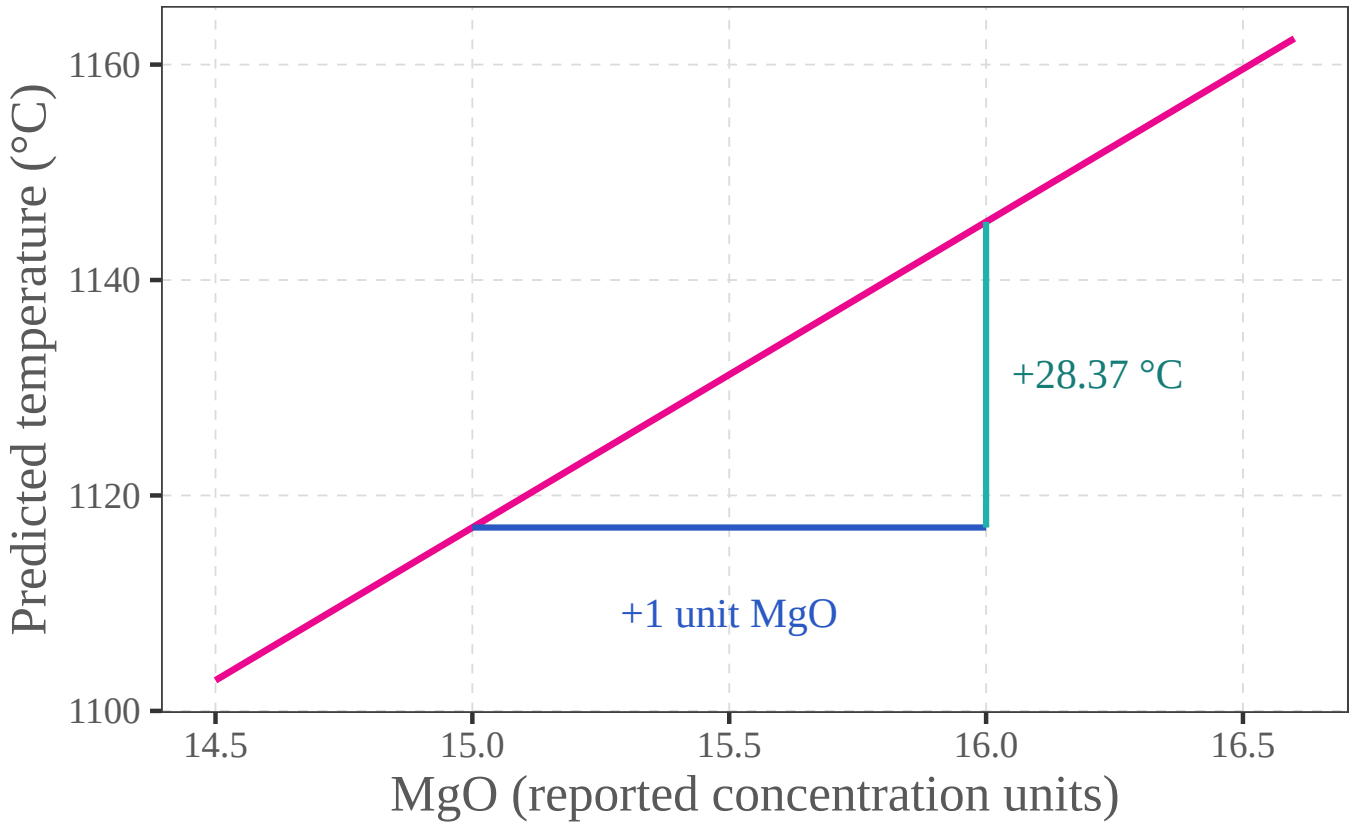
OLS Regression Results

Dep. Variable:	T_C	R-squared:	0.672
Model:	OLS	Adj. R-squared:	0.671
No. Observations:	1402	F-statistic:	954.5
Covariance Type:	nonrobust	Prob (F-statistic):	0.00

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	576.3177	15.454	37.292	0.000	546.002	606.633
MgO_Cpx	28.3747	0.768	36.954	0.000	26.868	29.881
Al2O3_Cpx	25.1036	0.821	30.583	0.000	23.493	26.714
TiO2_Cpx	-18.6177	4.032	-4.618	0.000	-26.527	-10.709

Notes:
[1] The coefficient standard errors assume constant error variance and uncorrelated errors across observations.

Interpreting multiple-regression coefficients



Held fixed: Al_2O_3 = 5.20; TiO_2 = 0.83 (reported units)

$$\Delta \hat{T} = 28.37 \, \Delta \mathrm{MgO}$$

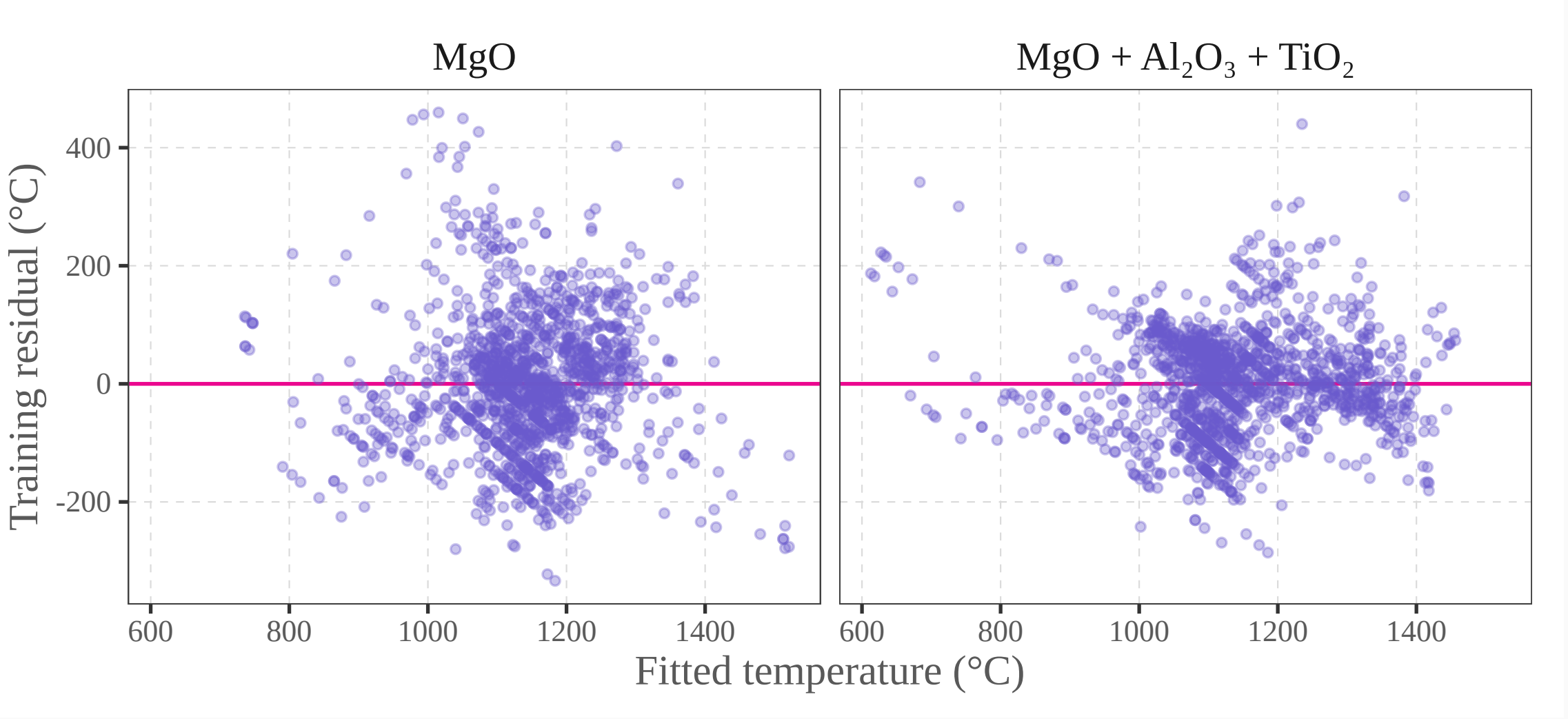
Holding Al_2O_3 and TiO_2 fixed, one more unit of reported MgO corresponds to **28.37 °C higher predicted mean temperature**.

The other slopes follow the same rule:

- Al_2O_3: **+25.10 °C** per reported unit.
- TiO_2: **−18.62 °C** per reported unit.

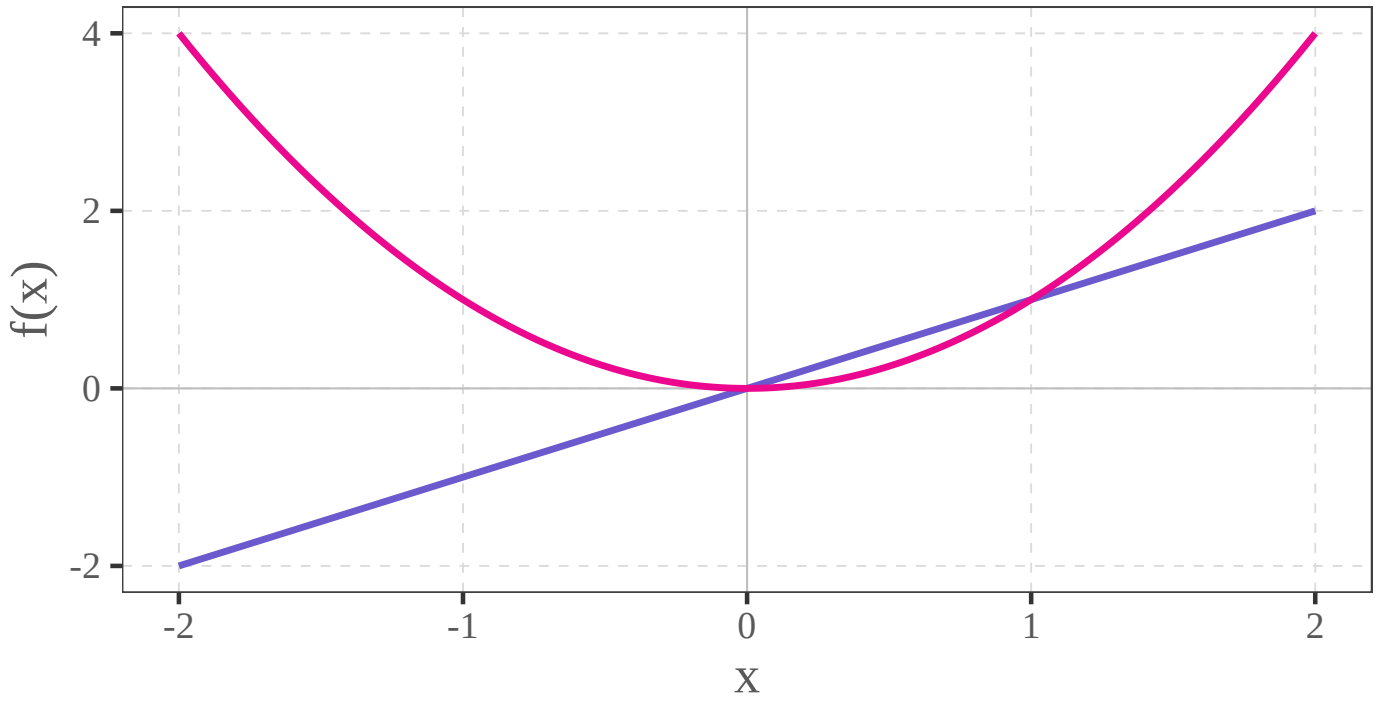
Each comparison holds the other two oxides fixed.

Model diagnostics: what remains unexplained?



Look for **curvature, changing spread, clusters, and unusual observations**. Zero average residual is automatic for OLS with an intercept; it does not establish model adequacy.

From straight lines to curves



A straight line changes at a constant rate.

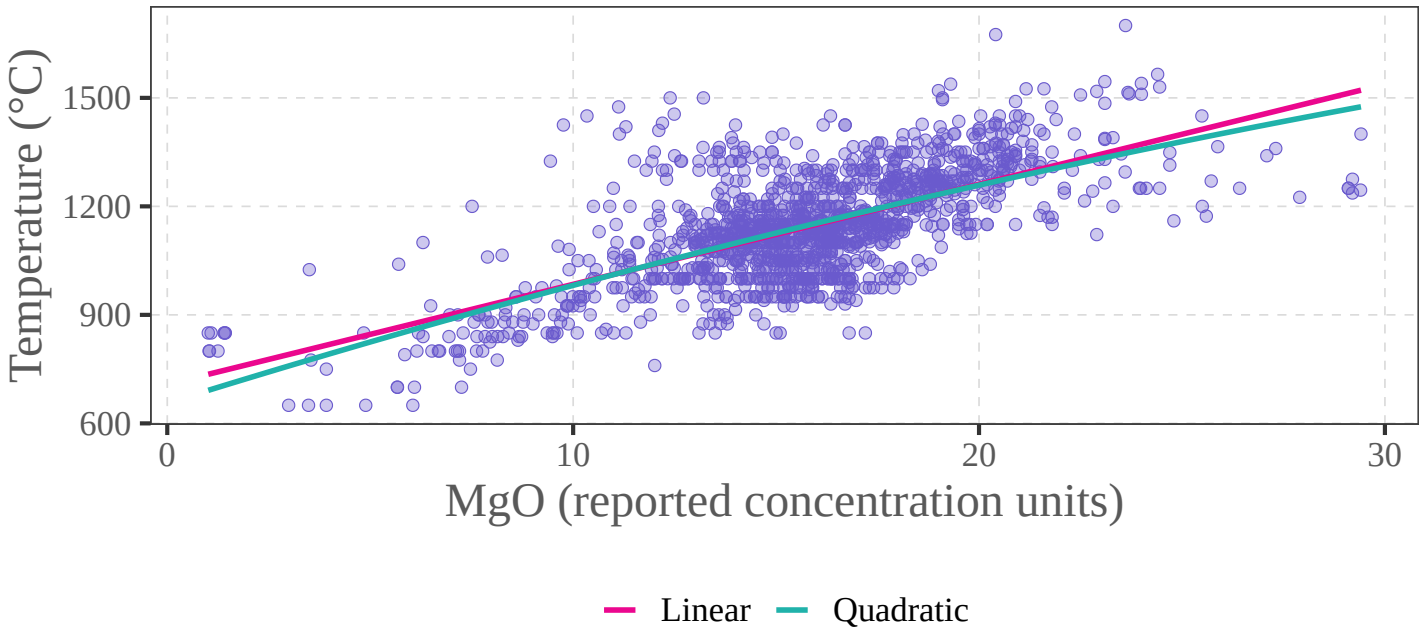
$$f(x) = x^2$$

Squaring x gives us another predictor. Combining x and x^2 lets a fitted mean bend.

Mathematical illustration, not CPX observations.

Polynomial regression

$$T = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon, \qquad x = \mathrm{MgO}.$$



Training RMSE: linear 114.7 °C; quadratic 114.5 °C

Curved in x, but linear in the coefficients: each β_j multiplies a known predictor.

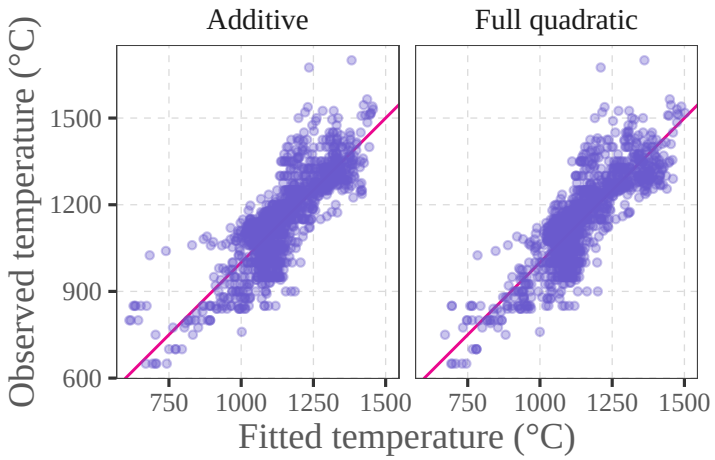
Here, adding curvature barely changes training error. This does not establish better predictions.

The curve is an empirical approximation, not a physical law. The test set is not used to choose the polynomial degree.

A more flexible model for three oxides

$$T = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_1^2 + \beta_5 x_2^2 + \beta_6 x_3^2 + \beta_7 x_1 x_2 + \beta_8 x_1 x_3 + \beta_9 x_2 x_3 + \varepsilon.$$

$x_1 = \mathrm{MgO}$, $x_2 = \mathrm{Al_2O_3}$, $x_3 = \mathrm{TiO_2}$, in their reported concentration units.



Training R^2: additive 0.672; quadratic 0.698

10 coefficients instead of 4.

Squared terms allow curvature. Products (interaction) let the relationship with one oxide depend on another oxide.

Still linear in the coefficients.

Fit the quadratic model in Python

Fit the three-oxide model with linear terms, squared terms, and interactions.

```
from cpx_lesson import load_data
import statsmodels.formula.api as smf

train, _ = load_data()
formula = (
    "T_C ~ (MgO_Cpx + Al2O3_Cpx + TiO2_Cpx)**2"
    " + I(MgO_Cpx**2) + I(Al2O3_Cpx**2) + I(TiO2_Cpx**2)"
)
quadratic = smf.ols(formula, data=train).fit()
print(f"Training R² = {quadratic.rsquared:.3f}")
```

In the formula, `(…)**2` includes pairwise interactions; `I(x**2)` adds a squared predictor. Training fit alone does not establish predictive benefit.

Summarize the quadratic model

Use `quadratic.summary(slim=True)` to inspect the full three-oxide quadratic.

```
from cpx_lesson import load_data
import statsmodels.formula.api as smf
from IPython.display import display

train, _ = load_data()
formula = (
    "T_C ~ (MgO_Cpx + Al2O3_Cpx"
    " + TiO2_Cpx)**2"
    " + I(MgO_Cpx**2)"
    " + I(Al2O3_Cpx**2)"
    " + I(TiO2_Cpx**2)"
)
quadratic = smf.ols(
    formula, data=train
).fit()
display(quadratic.summary(slim=True))
```

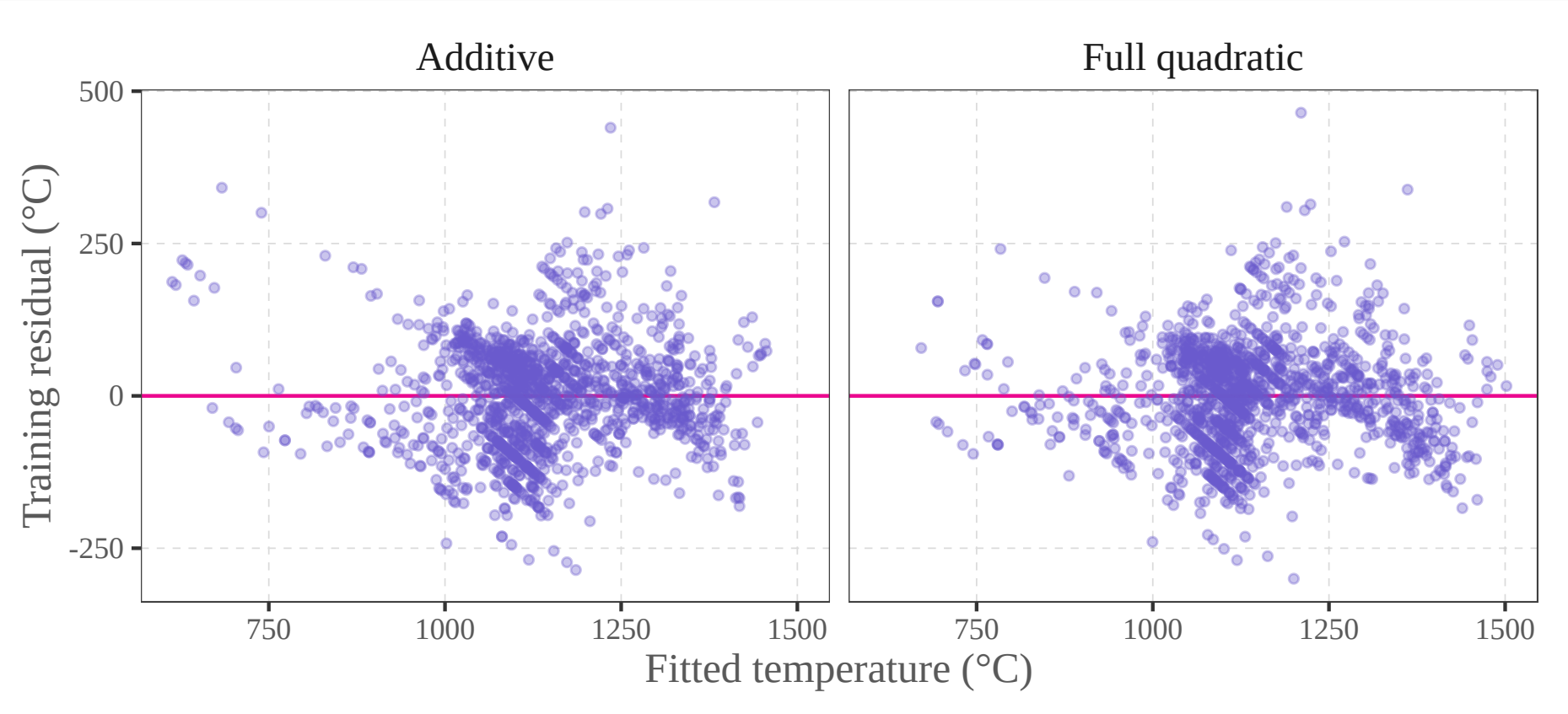
OLS Regression Results

Dep. Variable:	T_C	R-squared:	0.698
Model:	OLS	Adj. R-squared:	0.696
No. Observations:	1402	F-statistic:	358.0
Covariance Type:	nonrobust	Prob (F-statistic):	0.00

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	794.7026	40.242	19.748	0.000	715.761	873.644
MgO_Cpx	16.7772	3.903	4.298	0.000	9.120	24.434
Al2O3_Cpx	-29.1115	6.522	-4.464	0.000	-41.906	-16.317
TiO2_Cpx	-50.3671	23.114	-2.179	0.029	-95.710	-5.024
MgO_Cpx:Al2O3_Cpx	1.7387	0.250	6.969	0.000	1.249	2.228
MgO_Cpx:TiO2_Cpx	2.9191	1.185	2.463	0.014	0.595	5.244
Al2O3_Cpx:TiO2_Cpx	0.4269	1.532	0.279	0.781	-2.578	3.432
I(MgO_Cpx ** 2)	0.0730	0.107	0.684	0.494	-0.136	0.283
I(Al2O3_Cpx ** 2)	2.1780	0.249	8.764	0.000	1.690	2.666
I(TiO2_Cpx ** 2)	1.1636	2.932	0.397	0.692	-4.588	6.916

Notes:
[1] The coefficient standard errors assume constant error variance and uncorrelated errors across observations.
[2] The condition number is large, 5.8e+03. This might indicate that there are strong multicollinearity or other numerical problems.

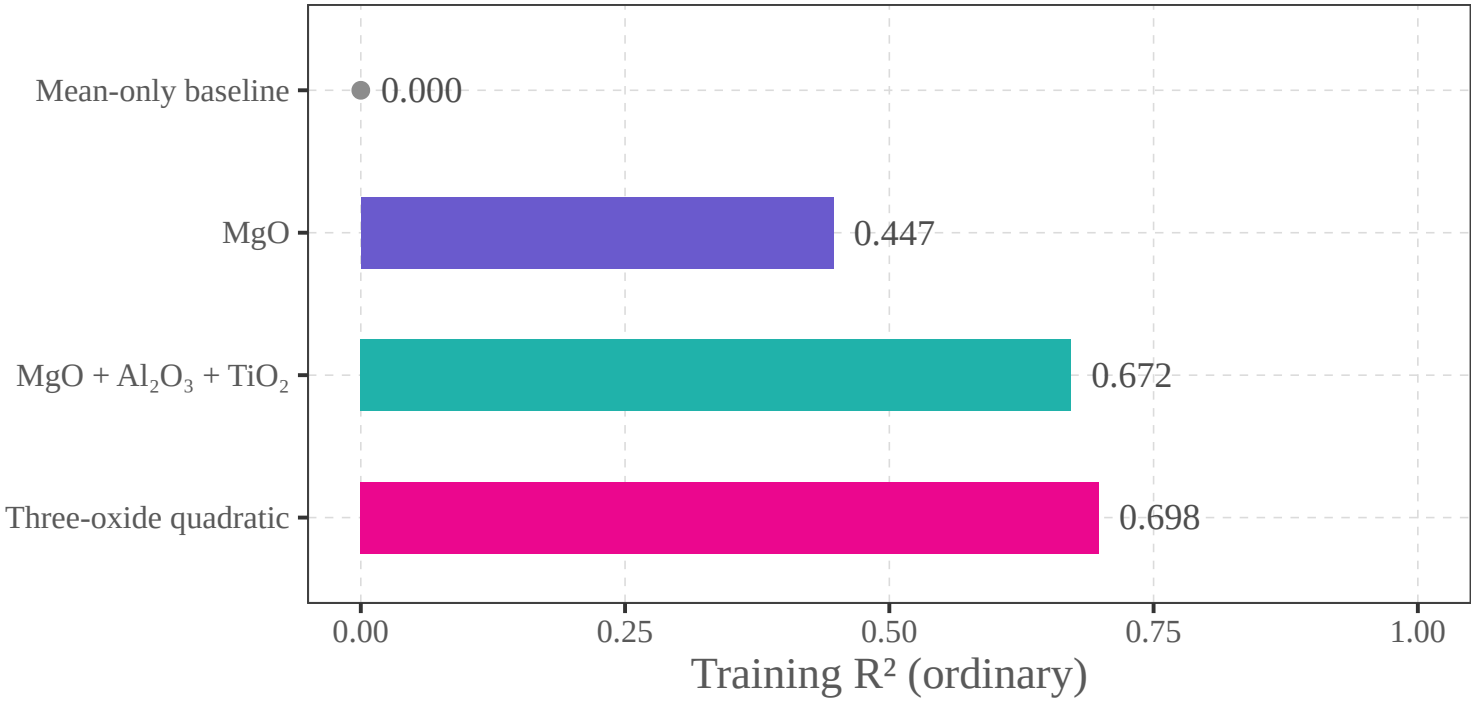
Model diagnostics: the quadratic model



Residual = observed − fitted temperature. Do squared terms and interactions remove systematic patterns?

Look again for **curvature, changing spread, clusters, and unusual observations**.

Adding predictors and R-squared



Same 1,402 training observations; each model includes an intercept

Adding predictors cannot decrease training R^2.

The larger model can recover the smaller one by setting the extra coefficients to **zero**. Least squares therefore cannot fit worse.

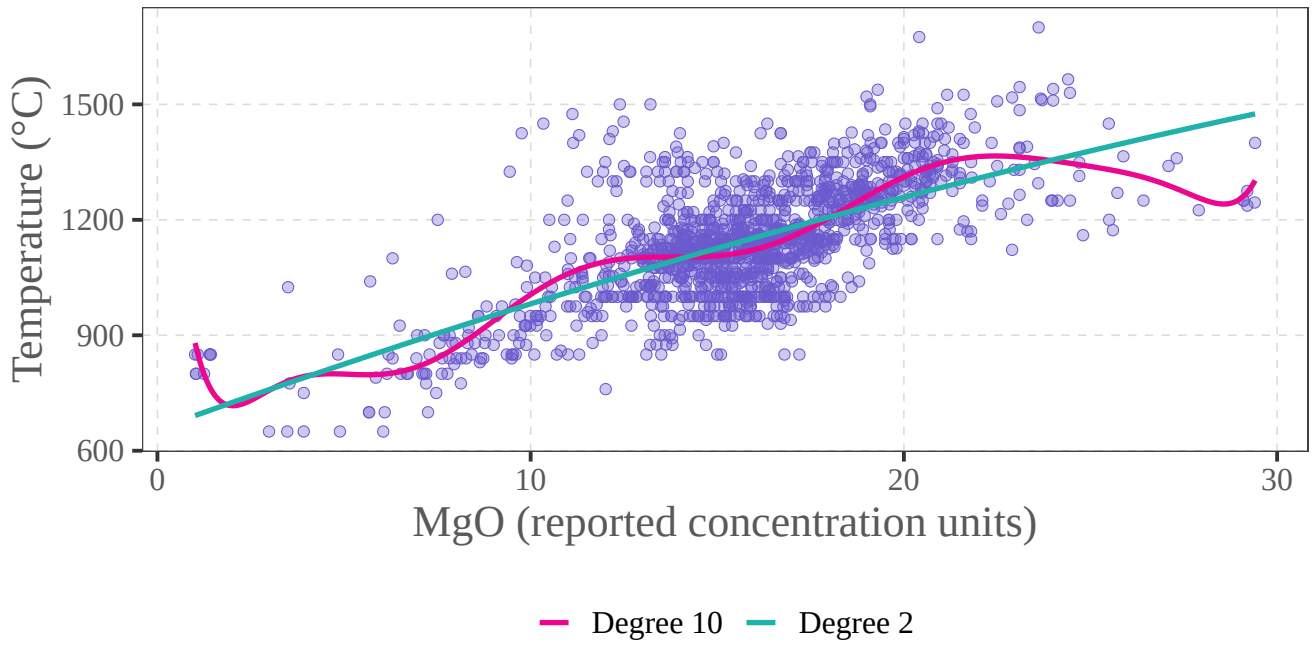
The models must use the **same observations**, include an intercept, and retain the earlier predictors.

R^2 can stay unchanged. This rule concerns **ordinary** R^2, not adjusted R^2.

> Higher training R^2 alone does not establish better predictions.

A degree-10 polynomial using MgO

$$T = \beta_0 + \sum_{k=1}^{10} \beta_k z^k + \varepsilon.$$



Training RMSE: degree 2 114.5 °C; degree 10 108.6 °C

11 coefficients, using only MgO.

z: MgO rescaled to $[-1, 1]$ for numerical stability. This remains a degree-10 polynomial in MgO.

Extra powers allow a more flexible curve.

Training observations only. Lower training error does not guarantee better test predictions.

Fit the degree-10 model in Python

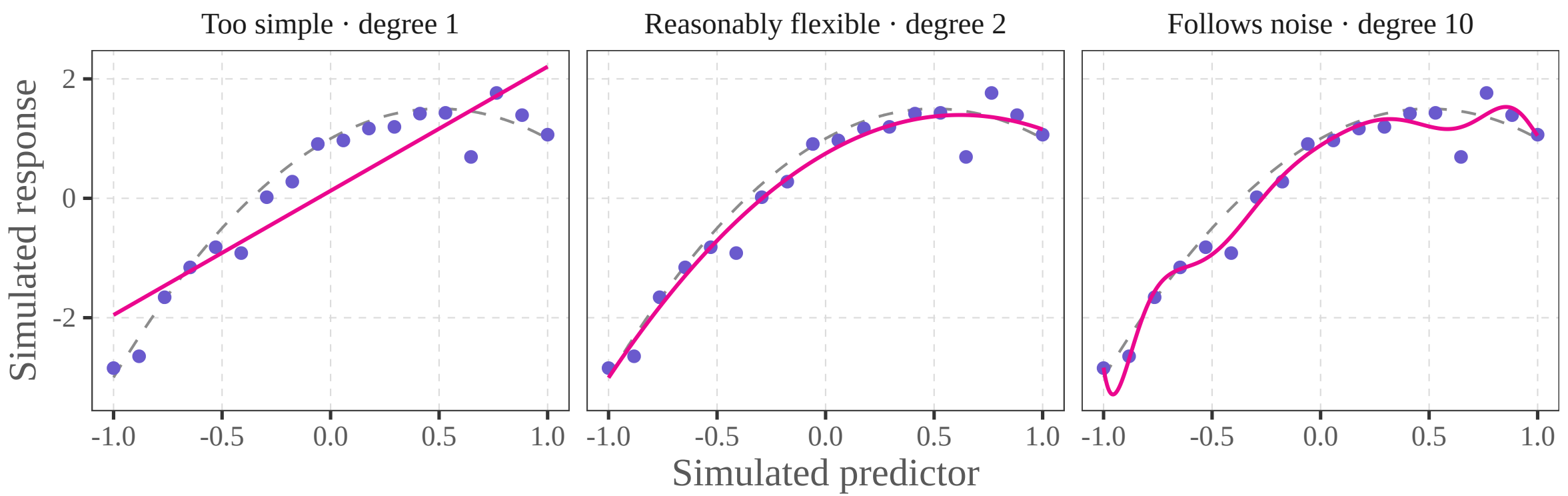
Rescale MgO, then fit its powers from degree 1 through degree 10.

```
from cpx_lesson import load_data
import statsmodels.formula.api as smf

train, _ = load_data()
lo, hi = train.MgO_Cpx.min(), train.MgO_Cpx.max()
train["z"] = 2 * (train.MgO_Cpx - lo) / (hi - lo) - 1
terms = " + ".join(f"I(z**{k})" for k in range(1, 11))
poly10 = smf.ols("T_C ~ " + terms, data=train).fit()
print(f"Training R² = {poly10.rsquared:.3f}")
```

`range(1, 11)` supplies powers 1–10; the formula includes an intercept. Keep `lo` and `hi` to apply the same rescaling to future observations.

Overfitting



Simulated data · pink: fitted model · dashed grey: generating mean

Simulated data, not CPX: greater flexibility reduces training error here, but can also follow noise. Smaller training error does not guarantee better predictions.

> Must a degree-10 polynomial predict better than a degree-2 polynomial?

“All models are wrong, but some are useful.”

George E. P. Box (1979), ***Robustness in the Strategy of Scientific Model Building***, in *Robustness in Statistics*, pp. 201–236; quotation on p. 202.



A straight-line mean

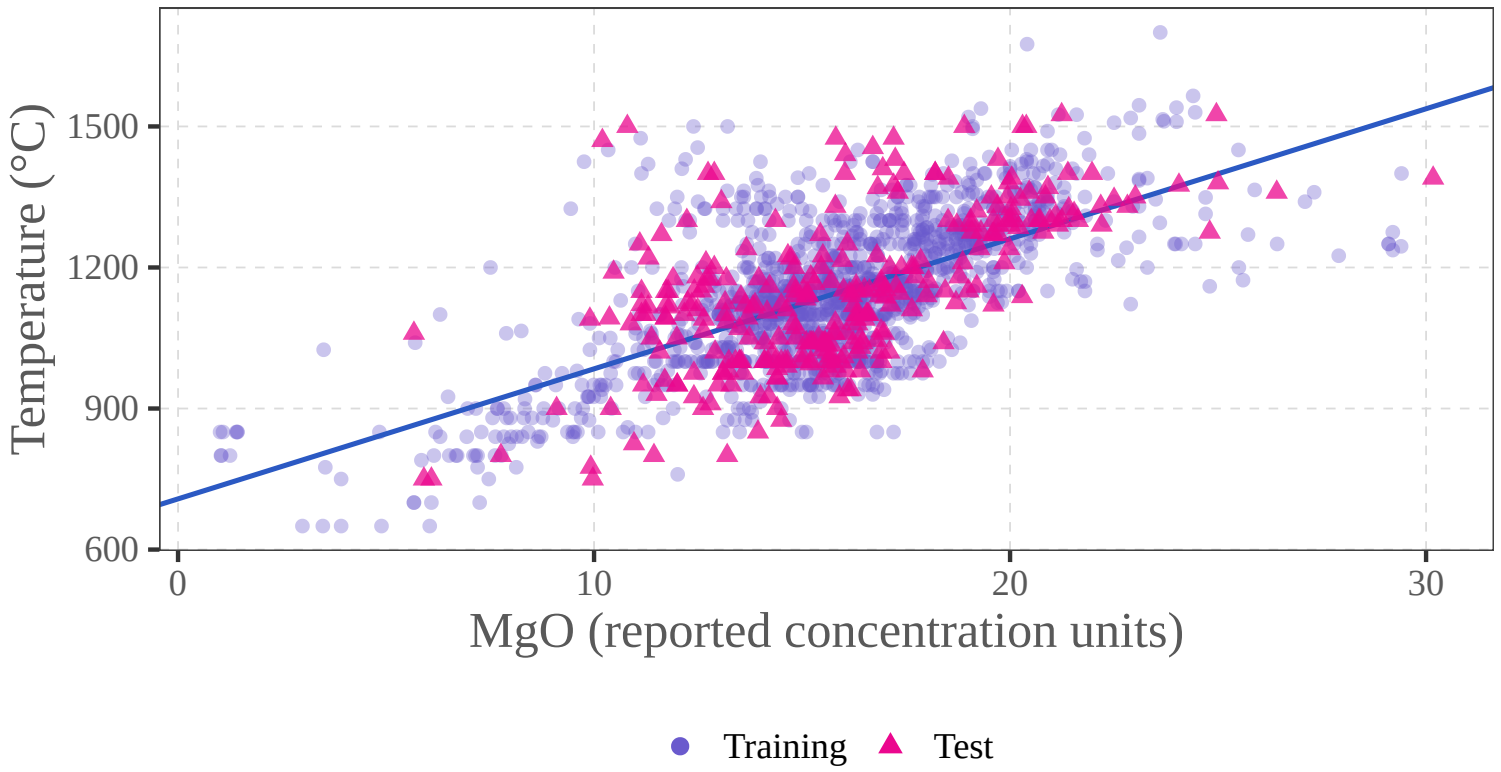
Summarizes an average trend with few coefficients.

A polynomial mean

Can describe curvature, while still omitting experimental processes.

> Usefulness depends on the scientific purpose, the supported domain, and the acceptable prediction error.

Training and test datasets



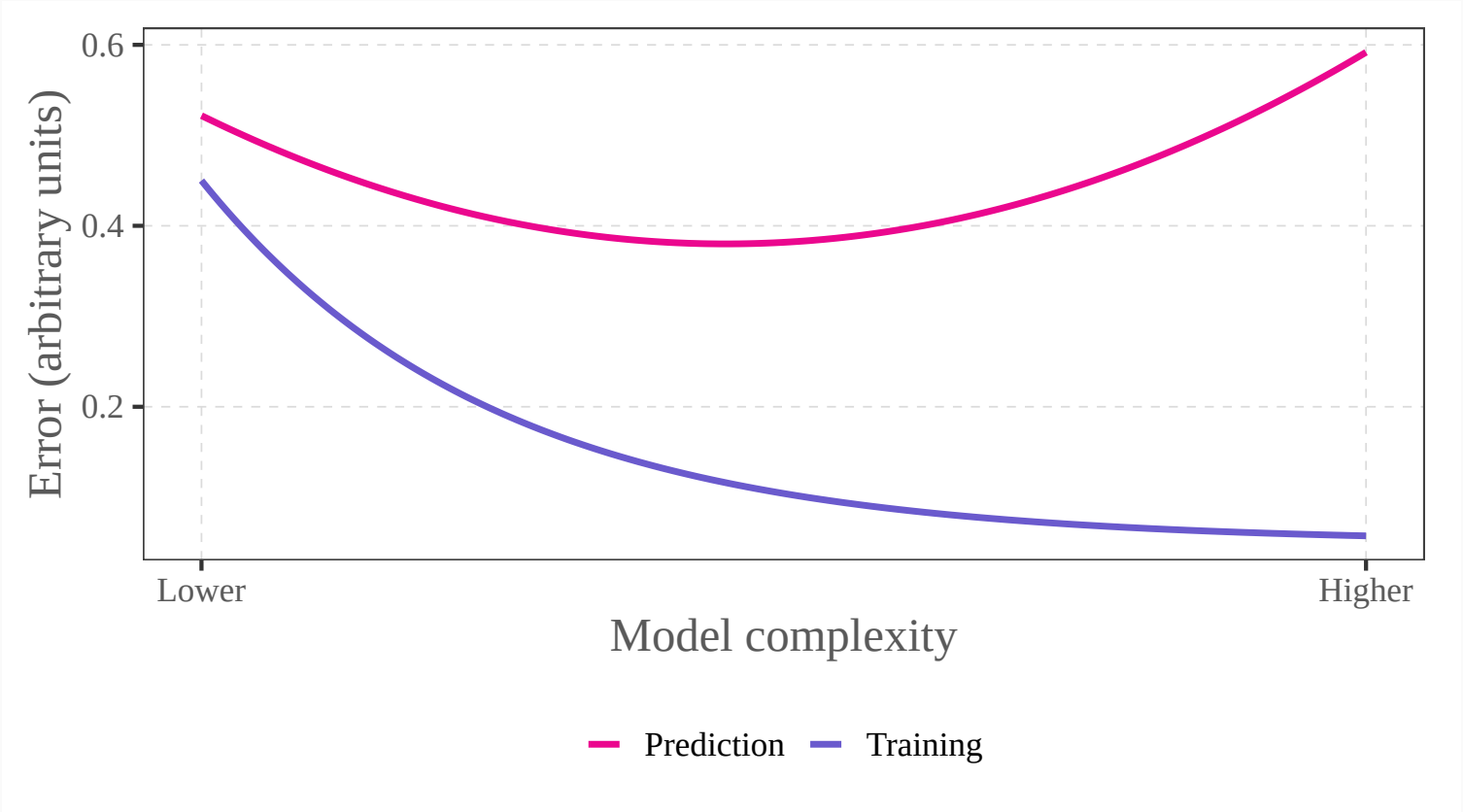
Blue line: MgO model fitted on training observations

Training: estimate the model's coefficients.

Test: predict without refitting, then compare with the known experimental temperatures.

Good fit on the observations used to estimate a model does not guarantee accurate predictions for other observations.

Model complexity versus out-of-sample error



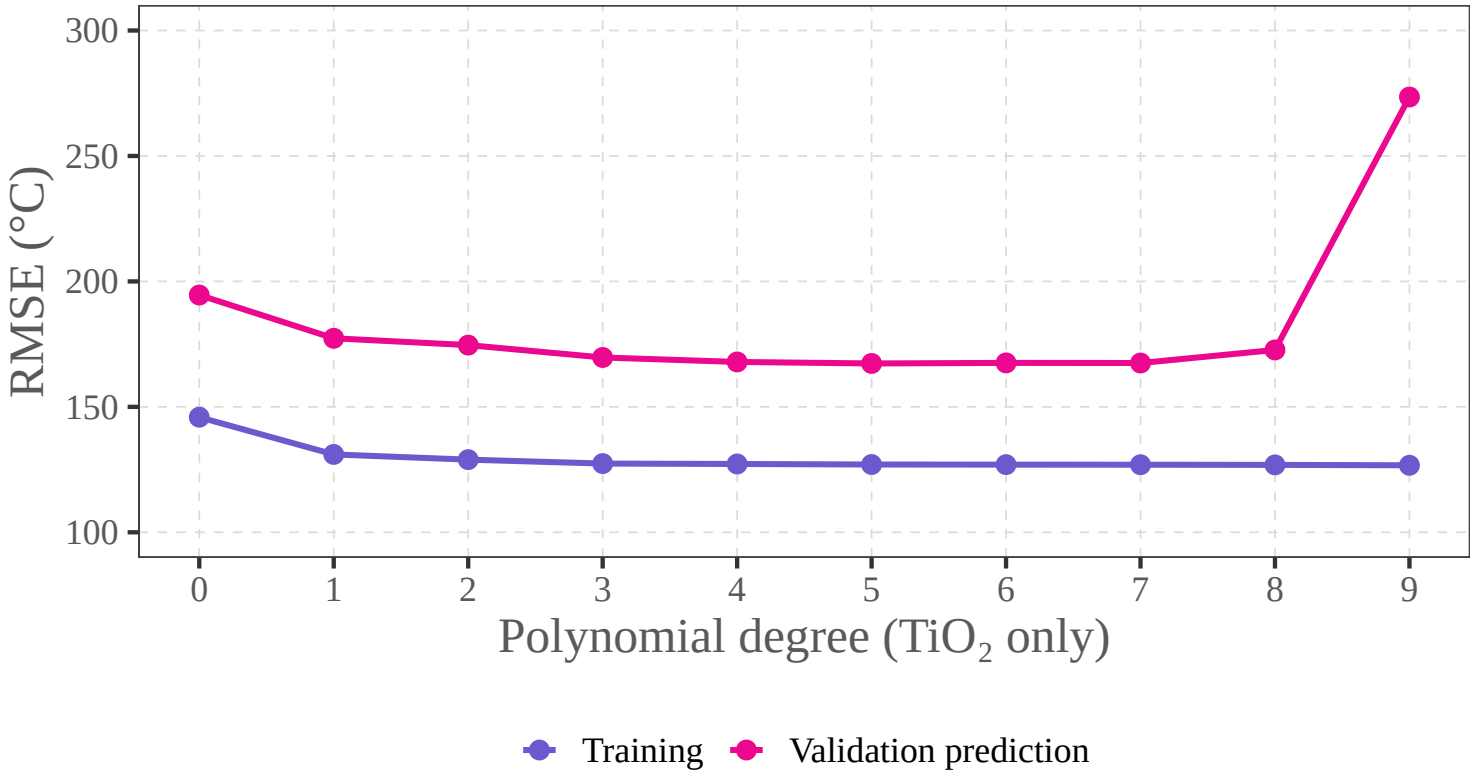
Too little flexibility can miss a real pattern.

More flexibility can also follow random variation.

Schematic only: a U-shaped prediction-error curve is illustrative, not guaranteed.

Keep the test set for the final assessment; do not repeatedly use it to choose polynomial complexity.

Model complexity: an example with TiO_2



Actual CPX data · fixed validation subset; final test unused

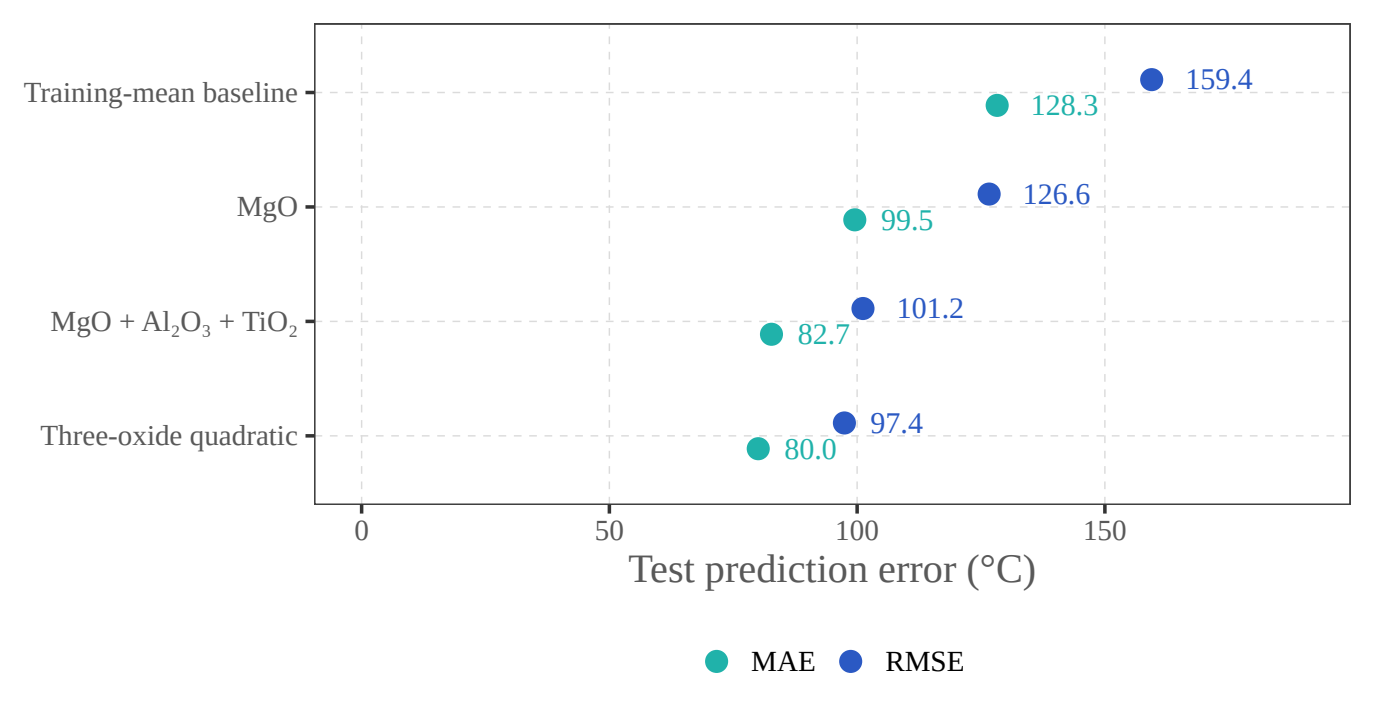
Training error decreases, then levels off.

Validation error initially falls, then rises at high degrees.

Degree **0** predicts the fitting mean.

How accurate are our temperature predictions?

Models are fitted using training observations and evaluated using observations that were not used for fitting.



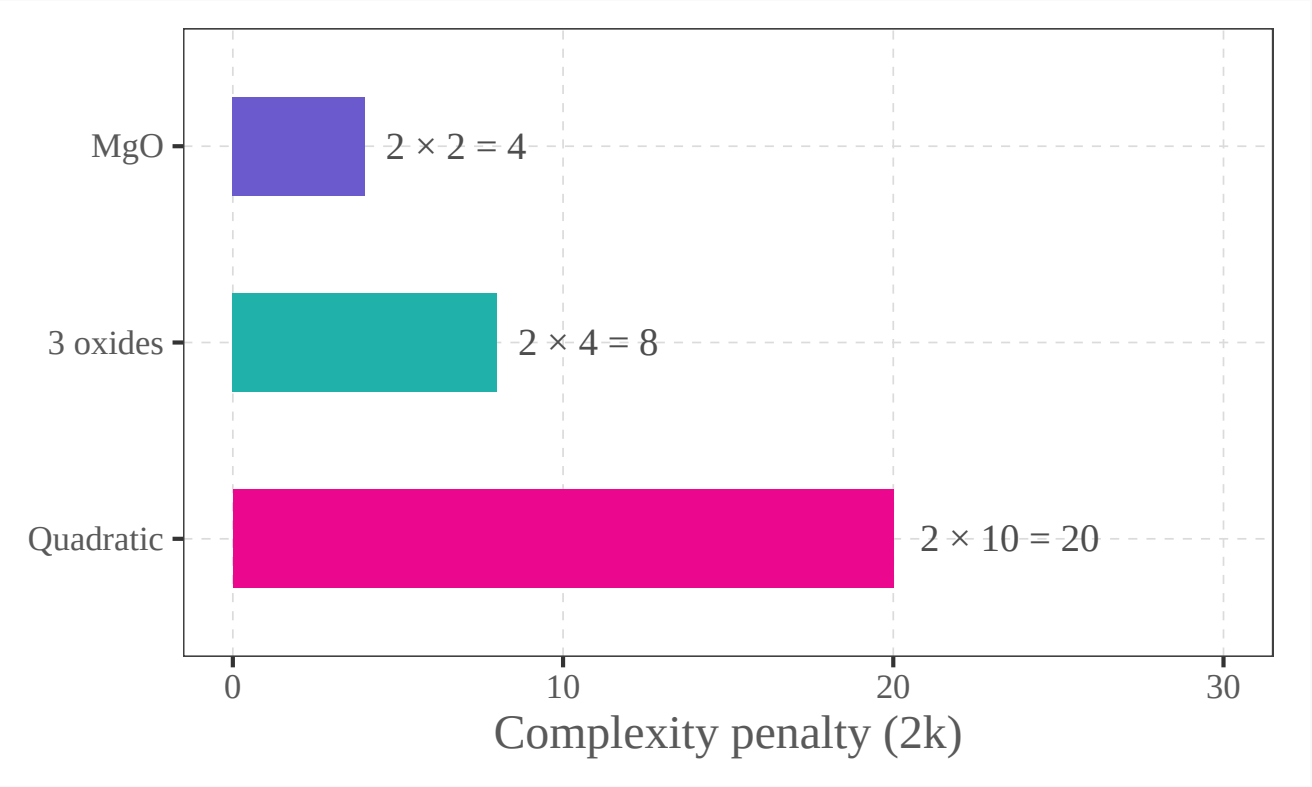
$$\mathrm{MAE} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} |T_i - \hat{T}_i|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (T_i - \hat{T}_i)^2}$$

n_{test}: number of test observations; T_i: measured temperature; $\hat{T}_i$: prediction. Both errors are in °C.

> Is a prediction error of approximately 100°C acceptable for geological applications?

AIC: balancing fit and model complexity



$$\text{AIC} = \underbrace{-2 \log \hat{L}}_{\text{fit}} + \underbrace{2k}_{\text{complexity}}$$

Better fit lowers the first term; more coefficients increase the penalty.

$\hat{L}$: maximized likelihood—how well the fitted model accounts for the data under its error assumptions.

k: fitted regression coefficients, including the intercept, using the `statsmodels` OLS convention here.

Compare the **same response and observations** under the same type of error model.

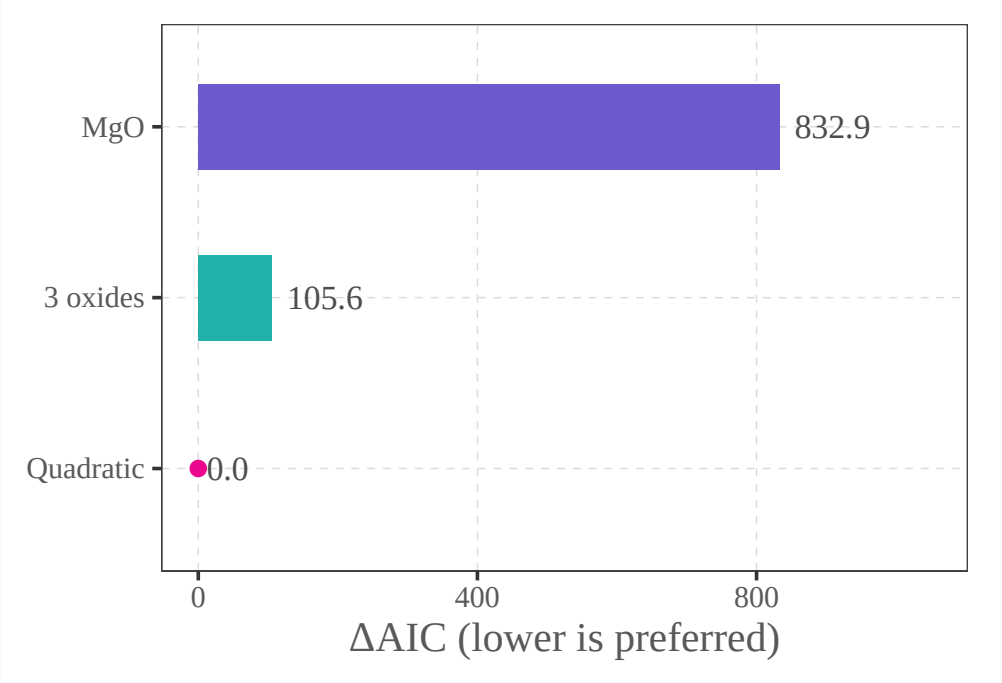
Lower AIC is preferred among the candidate models. It balances fit and complexity; it is not a prediction error in °C.

AIC for our three CPX models

Fit the same training observations; compare each fitted model's `.aic`.

```
from cpx_lesson import load_data
import statsmodels.formula.api as smf
import pandas as pd

train, _ = load_data()
linear = "MgO_Cpx + Al2O3_Cpx + TiO2_Cpx"
squares = ("I(MgO_Cpx**2) + I(Al2O3_Cpx**2)"
           " + I(TiO2_Cpx**2)")
formulas = {
    "MgO": "T_C ~ MgO_Cpx",
    "3 oxides": "T_C ~ " + linear,
    "Quadratic": f"T_C ~ ({linear})**2 + {squares}"
}
fits = {n: smf.ols(f, data=train).fit()
        for n, f in formulas.items()}
aic = pd.Series({n: m.aic for n, m in fits.items()})
print(aic.round(1).to_string())
```



$$\Delta\text{AIC} = \text{AIC} - \text{smallest AIC}.$$

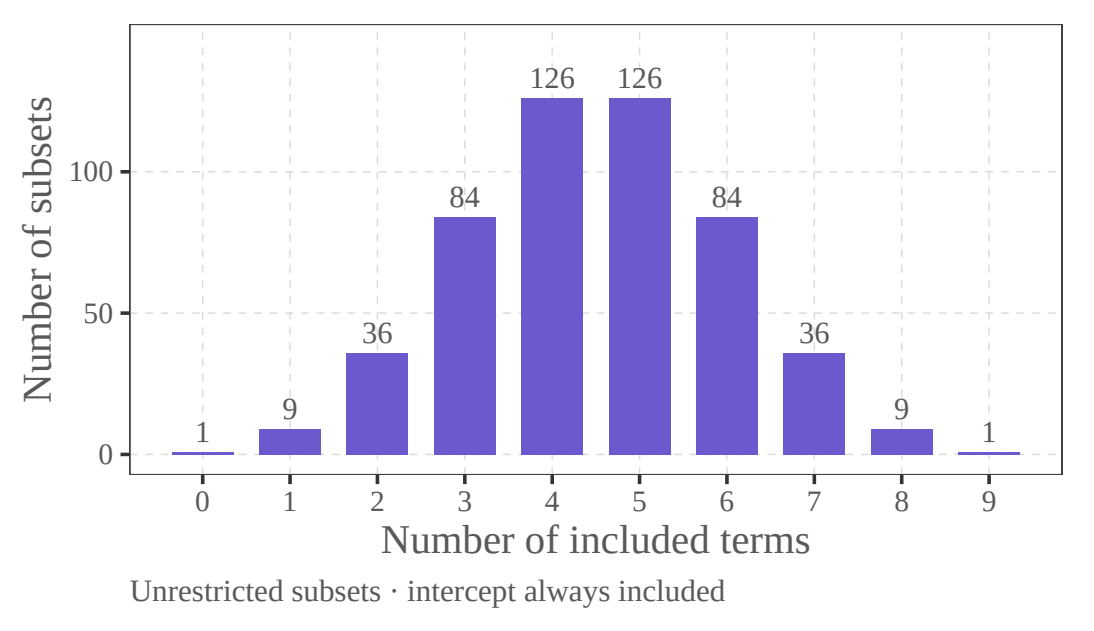
Here AIC favors the **quadratic**, under the working OLS assumptions.

Check predictive performance too.

Model selection: many models, one search strategy

Our three-oxide quadratic has **9 candidate terms**: 3 linear terms, 3 squares, and 3 products.

$$2^9 = 512 \text{ possible subsets}$$



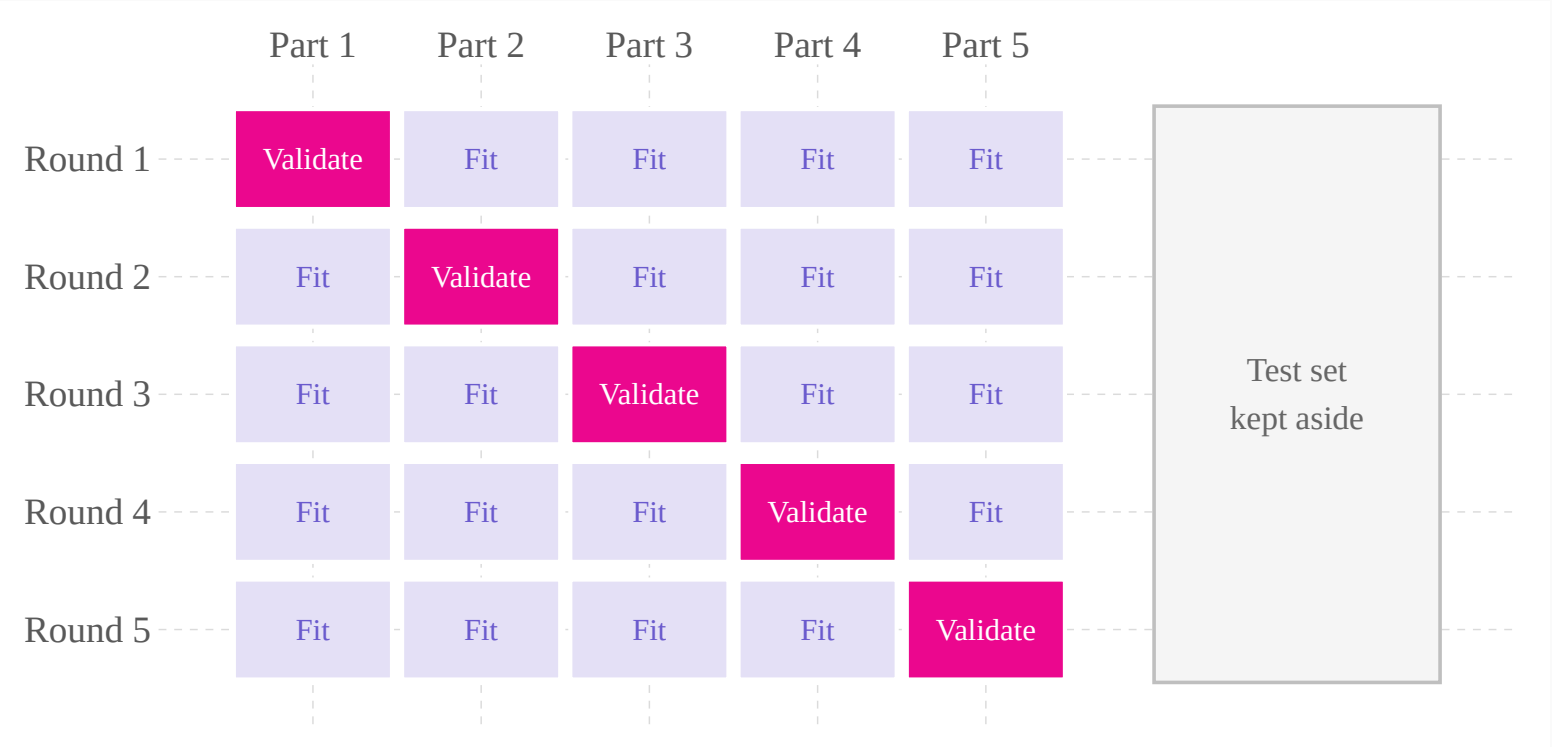
Forward selection using AIC

1. Start with an **intercept-only** model.
2. Try adding each allowed term, one at a time.
3. Keep the addition with the **lowest AIC**, if it lowers the current AIC.
4. Repeat until no allowed addition lowers AIC.

Choose candidate terms scientifically. Retain main terms when including their squares or interactions; such constraints reduce the search space.

> Forward search need not find the **lowest AIC among all subsets**. Check predictive performance with cross-validation next.

Cross-validation: rotate the validation observations



1. Split the training data into **five parts**, called folds.
2. Fit on four; predict the remaining part.
3. Repeat five times and combine the prediction errors.

Refit the model every time. Any learned preprocessing must also be fitted within each round.

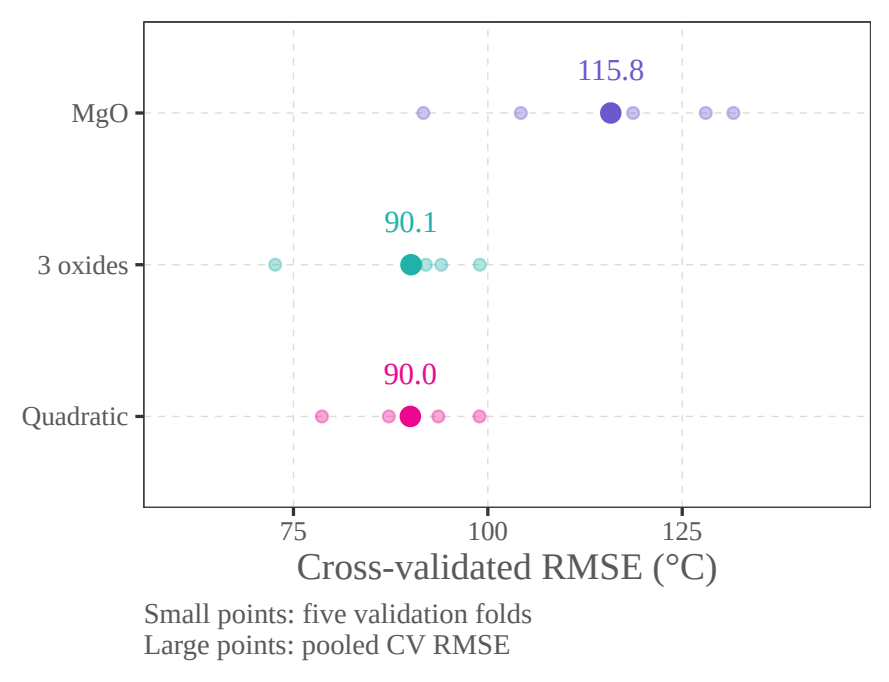
Keep the separate test set for the final assessment.

> Each observation is predicted by a model that **did not use it for fitting**.

Cross-validation for the CPX models

Use the same three formulas and five fixed folds inside the training data.

```
from cpx_lesson import load_data, cv_folds, selection_formulas
import statsmodels.formula.api as smf
import numpy as np, pandas as pd
train, _ = load_data()
fold = cv_folds(train)
scores = {}
for name, formula in selection_formulas().items():
    squared = []
    for v in range(5):
        fit = smf.ols(formula, train[fold != v]).fit()
        valid = train[fold == v]
        error = valid.T_C - fit.predict(valid)
        squared.extend(error ** 2)
    scores[name] = np.sqrt(np.mean(squared))
print(pd.Series(scores, name="CV RMSE").round(1))
```

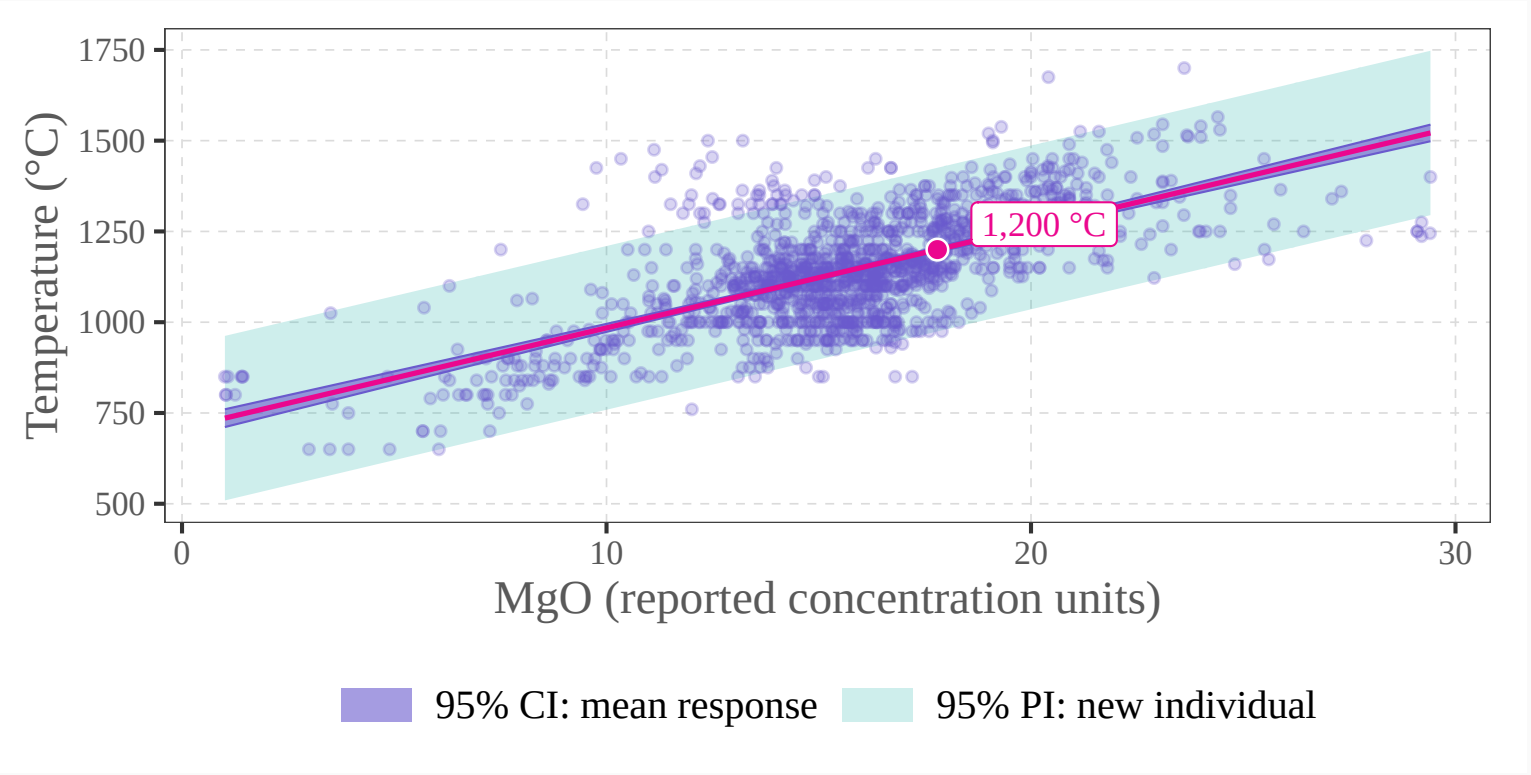


The three-oxide and quadratic models have **very similar CV error** here.

The supplied folds keep related observations together. The test set is unused in this calculation.

How uncertain is a temperature prediction?

A model predicts **1,200 °C**. How uncertain is this prediction?



Confidence interval: uncertainty about the **conditional mean** $E[T \mid \mathrm{MgO}]$.

Here: **1,193–1,207 °C**.

Prediction interval: for **one new individual observation** at the same MgO.

Here: **975–1,425 °C**.

The prediction interval is wider: it adds **residual variation** to uncertainty about the mean.

Classical model assumptions: a correct linear mean and independent, normally distributed errors with constant variance. Pointwise 95% bands from the training fit; no coverage claim for natural crystals or other experimental settings.

> Why can we estimate an average more precisely than predict one individual observation?

Computing confidence and prediction intervals

Classical 95% intervals at MgO x_0

$$\hat{T}_0 = \hat{\beta}_0 + \hat{\beta}_1 x_0, \qquad h_0 = \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

$$\text{CI}: \quad \hat{T}_0 \pm t_* \, s\sqrt{h_0}$$

$$\text{PI}: \quad \hat{T}_0 \pm t_* \, s\sqrt{1 + h_0}$$

$$s^2 = \frac{\sum_{i=1}^{n}(T_i - \hat{T}_i)^2}{n-2}, \qquad t_* = t_{n-2,\,0.975}.$$

n: training sample size; x_i: training MgO; $\bar{x}$: its mean. t_* is the 97.5th percentile of a t distribution with $n-2$ degrees of freedom.

The extra **1** adds **residual variance** s^2 when predicting an individual observation.

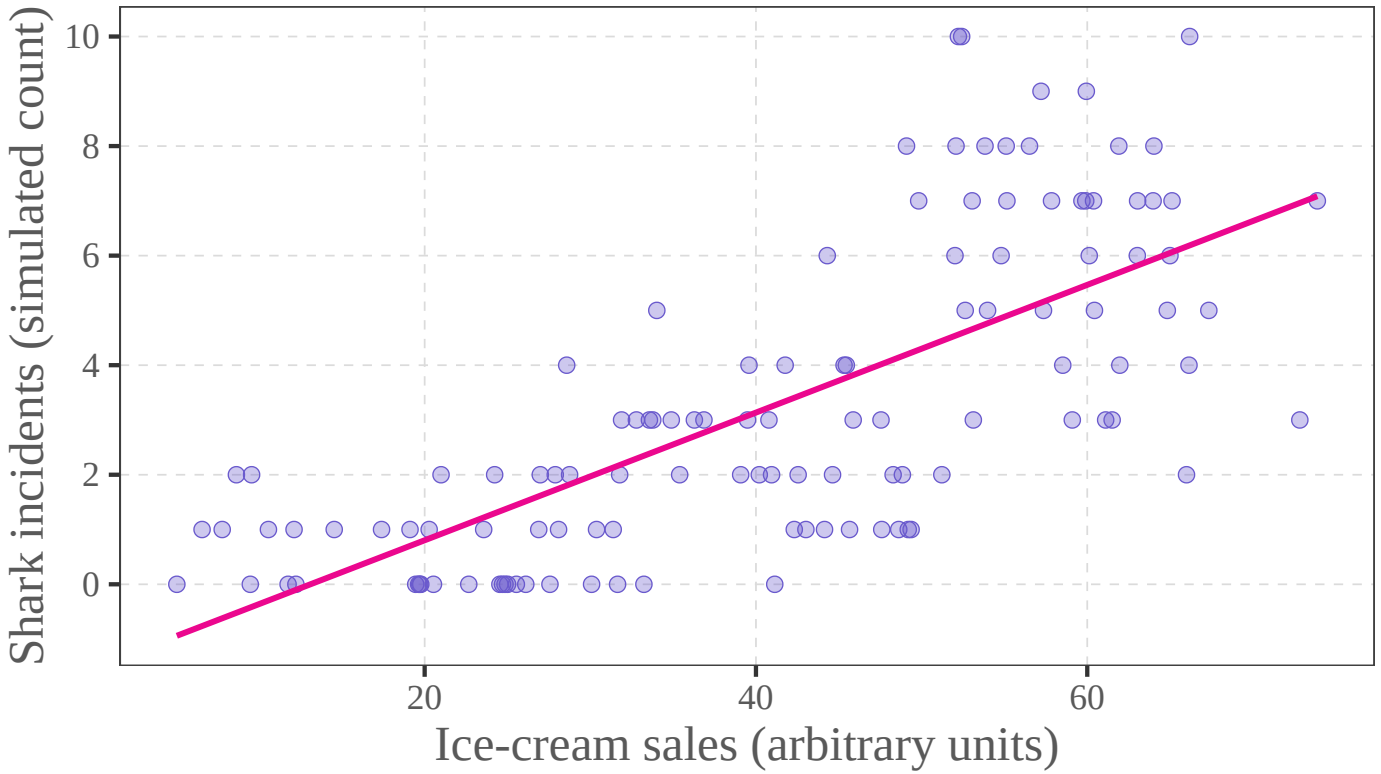
`mean_ci_*`: mean CI; `obs_ci_*`: individual PI. All in °C.

```python
from cpx_lesson import load_data
import statsmodels.formula.api as smf
import pandas as pd

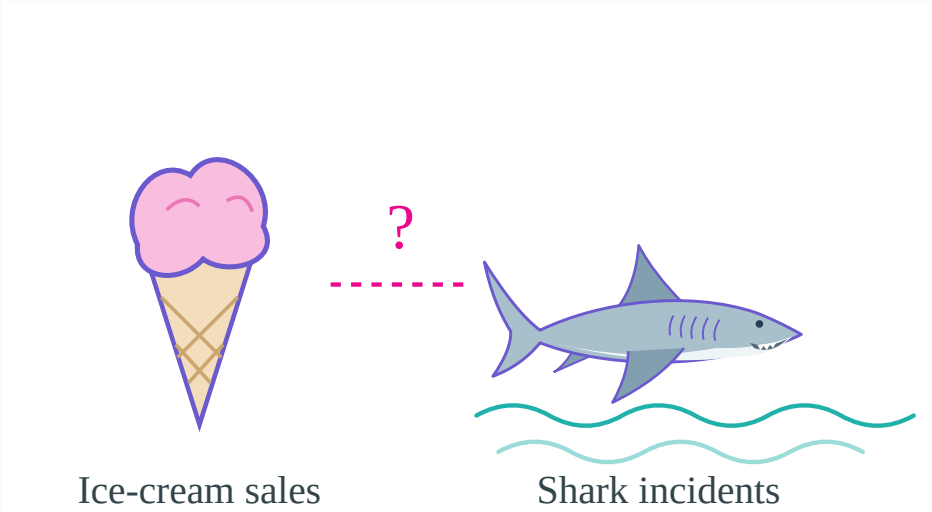
train, _ = load_data()
simple = smf.ols("T_C ~ MgO_Cpx", data=train).fit()
b0, b1 = simple.params
x0 = (1200 - b0) / b1  # fitted T = 1200 °C
new = pd.DataFrame({"MgO_Cpx": [x0]})
prediction = simple.get_prediction(new)
row = prediction.summary_frame(alpha=0.05).iloc[0]
keys = ["mean", "mean_ci_lower", "mean_ci_upper",
        "obs_ci_lower", "obs_ci_upper"]
print(row[keys].round(1).to_string())
```

Pointwise intervals under a correct linear mean and independent, normal errors with constant variance. Same experimental scope as the preceding slide.

Ice cream and sharks: association or cause?



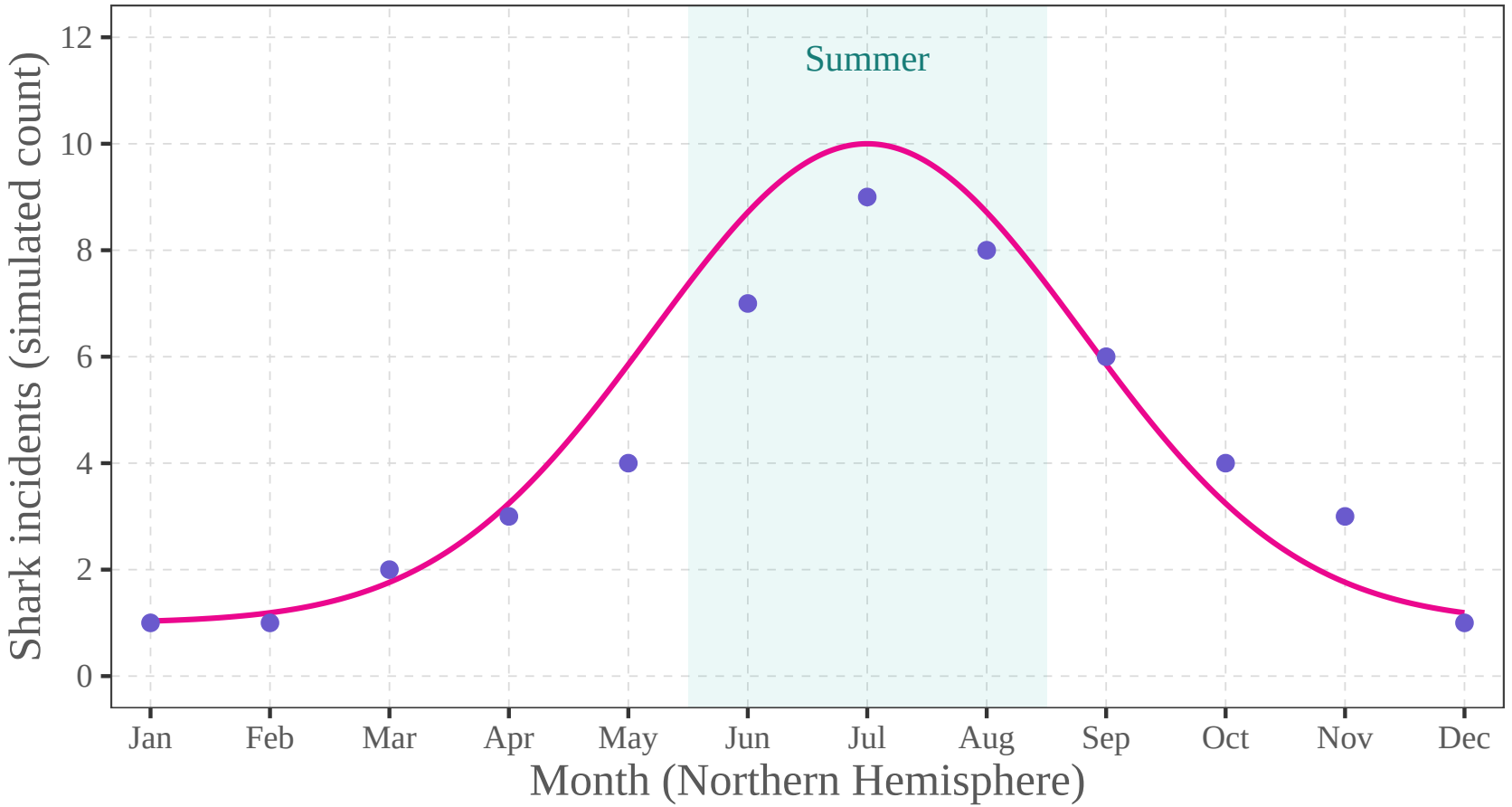
Simulated teaching example · not observed shark records



Does buying more ice cream **cause** more shark incidents?

A fitted line shows an association. It does not, by itself, explain why the variables move together.

Shark incidents through the year



Simulated teaching example · not observed shark records

What do you think?

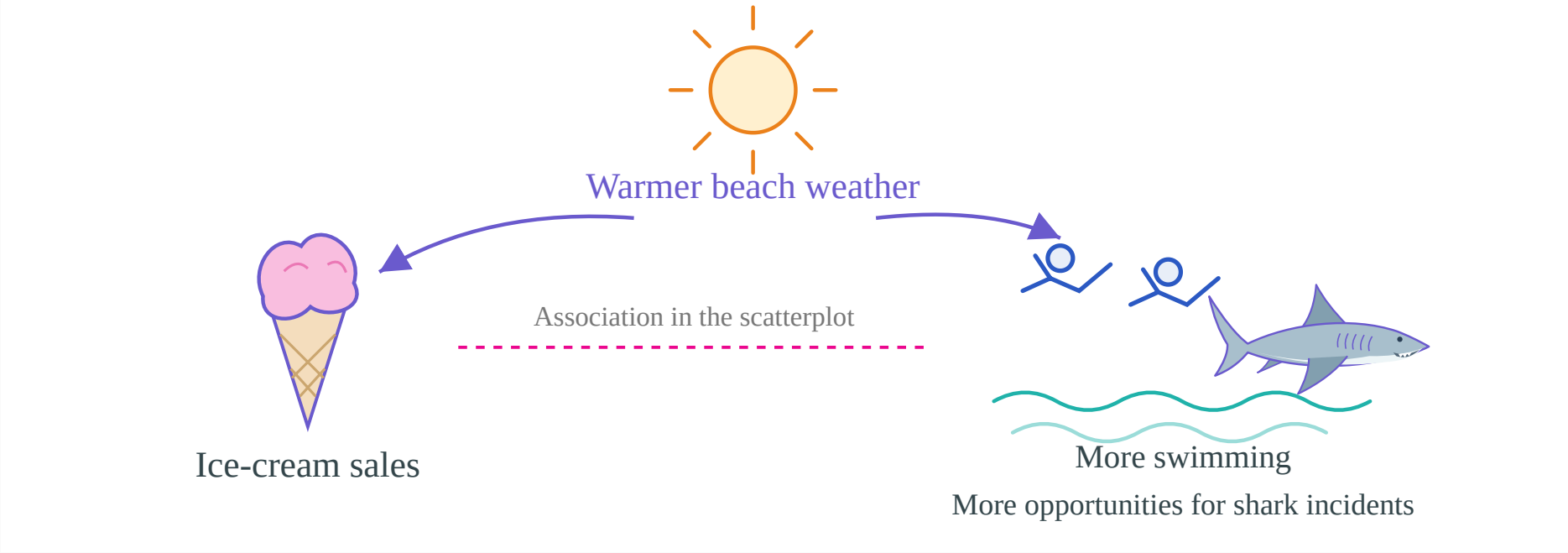
What else might vary between winter and summer?

Could it also affect ice-cream sales?

Points: simulated monthly counts.

Pink curve: the illustrative seasonal pattern used to generate them.

A common cause: warmer beach weather



Confounder: a common cause can create an association between two other variables.

Here: warmer weather can raise ice-cream demand and bring more people into the water.

Regression lesson: a coefficient—or a good prediction—does not establish causation.